

Relatório Intermédio — XCommerce: Monólito vs Microserviços

Mestrado em Engenharia e Gestão de Sistemas de Informação

PG60128- Rafael Rolim Prudente dos Santos

PG61472- Liandro Macedo da Cruz

PG60123- Miguel Ângelo Barroso Martins

PG61487- Tomás Castro Mendes de Sousa

1	<i>Introdução e Objetivo</i>	5
1.1	Enquadramento do Tema	5
1.2	Motivação	6
1.3	Sistema de Referência	6
1.4	Foco do Estudo	7
2	<i>Arquitetura Base do Sistema Atual (AS-IS)</i>	9
2.1	Visão Geral do Monólito	9
2.2	Stack Tecnológica	9
2.3	Organização por Camadas	10
2.4	Dependências Internas Principais	12
2.5	Principais Blocos Funcionais	13
	Bloco 1 — Autenticação e Gestão de Utilizadores	13
	Bloco 2 — Catálogo de Produtos	14
	Bloco 3 — Carrinho de Compras	15
	Bloco 4 — Encomendas	16
2.6	Modelo de Dados AS-IS	18
	Entidades e Relações	20
	Relações entre Entidades	21
	Dados Seed	22
2.7	Dependências Estruturais Relevantes	23
	Mapa de Dependências Componente → Entidade	23
	Dependência 1: Autenticação ↔ Domínio	24
	Dependência 2: Carrinho ↔ Catálogo	24
	Dependência 3: Encomenda ↔ Todos os Domínios	25
	Síntese das Dependências Cruzadas	25
3	<i>Workflows Analisados</i>	26
3.1	Critérios de Escolha dos Workflows	26
3.2	Workflow 1 — Adicionar Produto ao Carrinho	26
3.2.1	Descrição	26
3.3	Fluxo no Monólito (passo a passo)	27
3.4	Diagrama de Sequência	29
3.5	Dependências Cruzadas Identificadas	30

3.6	Workflow 2 — Consultar Encomendas de um Cliente	31
3.6.1	Descrição	31
3.7	Fluxo no Monólito (passo a passo)	31
3.8	Diagrama de Sequência	33
3.9	Dependências Cruzadas Identificadas	33
3.10	Workflow 3 — Finalizar Encomenda (Checkout)	34
3.10.1	Descrição	34
3.10.2	Fluxo no Monólito (passo a passo)	36
3.10.3	Diagrama de Sequência	38
3.11	Problemas de Consistência Evidenciados	38
3.12	Comparação com a Versão Microserviços (TO-BE)	39
3.13	Síntese das Dependências Observadas	41
3.13.1	Dependências Comuns	42
4	Arquitetura TO-BE (Microserviços).....	45
4.1	Variante TO-BE Adotada	45
4.1.1	Composição do Deployment	46
4.2	Justificação das Fronteiras	48
4.3	Papel do API Gateway	51
4.3.1	Autenticação JWT — AuthenticationFilter (GlobalFilter)	Error! Bookmark not defined.
4.4	APIs Externas (por Serviço).....	53
	Auth Service (porta 8081) — BD: <code>xcommerce_auth</code>	53
	Catalog Service (porta 8082) — BD: <code>xcommerce_catalog</code>	54
	Cart Service (porta 8083) — BD: <code>xcommerce_cart</code>	55
	Order Service (porta 8084) — BD: <code>xcommerce_orders</code>	56
	Inventory Service (porta 8085) — BD: <code>xcommerce_inventory</code>	58
	User Service (porta 8086) — BD: <code>xcommerce_users</code>	58
5	Objetivos, Questões de Investigação e Hipóteses	60
5.1	Objetivos do Estudo	60
5.2	Questões de Investigação	60
	Mapeamento QI → Objetivos.....	61
5.3	Hipóteses.....	62
	Mapeamento Hipóteses → QI.....	63
6	Dados, Consistência e Migração	64
6.1	Tabela de Ownership dos Dados.....	64
	Comparação com o Monólito	65
6.2	6.2 Dependências de Dados	66
	JOINS que deixaram de ser locais	66
6.3	Risco de Inconsistência	68

Cenário 1 — Stock (risco alto)	69
Cenário 2 — Encomendas (risco médio)	70
Cenário 3 — Carrinho (risco baixo)	70
Cenário 4 — Dados de utilizador duplicados (risco médio)	70
Cenário 5 — Pagamento assíncrono (risco controlado)	71
Resumo de Riscos	71
6.4 Plano de Migração Incremental	73
Fase 1 — Auth Service.....	73
Fase 2 — Catalog Service.....	73
Fase 3 — Cart Service	74
Fase 4 — Order Service + Inventory Service	74
Fase 5 — Payment, Notification e User Service	75
Fase 6 — API Gateway + Observabilidade	75
7 Metodologia Experimental e Testes Iniciais	77
7.1 Ambiente Experimental	77
7.2 Ferramentas.....	77
7.3 Cenários de Teste	78
7.4 Variáveis Controladas.....	79
7.5 Métricas	79
7.6 Testes Operacionais Já Realizados	80
8 Resultados Preliminares.....	82
8.1 Evidências Recolhidas	82
8.2 Comportamento do Monólito Perante Falha da Base de Dados	82
8.3 Testes por Workflow	83
WF1 — Adicionar Produto ao Carrinho.....	83
WF3 — Checkout.....	85
9 Discussão Inicial.....	86
9.1 Onde a Decomposição É Mais Sólida	86
9.2 Onde Surgem Dificuldades.....	86
9.3 Acoplamento Residual.....	87
9.4 Limitações do Estudo	87
9.5 Referências	88

Introdução e Objetivo

Enquadramento do Tema

No contexto atual dos sistemas de informação, as arquiteturas distribuídas baseadas em microsserviços têm vindo a assumir um papel central no desenvolvimento de aplicações escaláveis, resilientes e de fácil evolução. Em contraste com as arquiteturas monolíticas tradicionais, onde toda a lógica aplicacional é implementada e executada como uma única unidade, os microsserviços promovem a decomposição do sistema em serviços independentes, cada um responsável por uma funcionalidade específica e com capacidade de evolução autónoma.

Esta abordagem permite responder de forma mais eficaz a requisitos modernos de engenharia de software, nomeadamente em termos de escalabilidade, disponibilidade, deployability e custo operacional, mas introduz também novos desafios, como a complexidade de comunicação entre serviços, gestão de consistência de dados e aumento da sobrecarga operacional.

Neste contexto, a presente unidade curricular de Arquiteturas de Sistemas de Informação Distribuídos (ASID) tem como objetivo estudar, conceber e avaliar arquiteturas baseadas em microsserviços, articulando fundamentação teórica com validação empírica. O foco do projeto incide na análise de decisões arquiteturais, na comparação de alternativas e na avaliação do impacto dessas decisões em atributos de qualidade relevantes, suportada por experimentação controlada.

O presente trabalho enquadra-se na modalidade b) — refactoring arquitetural de uma aplicação monolítica para uma arquitetura baseada em microsserviços. Como ponto de partida, foi utilizada a aplicação XCommerce Monolithic, uma plataforma de e-commerce desenvolvida em Java/Spring Boot, estruturada segundo uma arquitetura monolítica com organização interna orientada a serviços. Esta aplicação integra componentes como persistência relacional (PostgreSQL), cache (Redis), comunicação via REST e autenticação baseada em JWT, sendo executada como um único artefacto aplicacional.

Com base neste sistema inicial, foi realizada a sua decomposição numa arquitetura de microsserviços composta por oito serviços de negócio independentes, complementados por um API Gateway, responsável pela centralização do acesso externo ao sistema. Esta transformação teve como objetivo permitir uma análise estruturada dos benefícios e trade-offs associados à adoção de uma arquitetura distribuída, nomeadamente ao nível da granularidade dos serviços, isolamento de responsabilidades e impacto na comunicação inter-serviços.

Assim, o trabalho centra-se na avaliação crítica da transição de uma arquitetura monolítica para microsserviços, procurando responder à questão fundamental:

em que medida a decomposição arquitetural contribui para melhorar atributos de qualidade como performance, availability, deployability e cost, face à solução monolítica original?

Motivação

A migração de arquiteturas monolíticas para microsserviços tem ganho relevância como resposta às limitações dos sistemas tradicionais face a requisitos modernos como escalabilidade, resiliência e evolução contínua. No caso do sistema XCommerce Monolithic, esta transição revela-se particularmente pertinente, dado que a aplicação foi concebida como um único artefacto executável, o que condiciona a sua flexibilidade e adaptação a diferentes padrões de carga.

Do ponto de vista da escalabilidade, o monólito obriga à replicação integral da aplicação, mesmo quando apenas certos componentes (como o catálogo) necessitam de maior capacidade. Paralelamente, a manutenção é dificultada pelo elevado acoplamento interno, uma vez que todos os módulos partilham o mesmo código, base de dados e ciclo de deployment, aumentando o risco de efeitos colaterais em alterações aparentemente isoladas.

Ao nível do isolamento de falhas, a dependência de componentes centralizados, como a base de dados PostgreSQL, implica que uma falha comprometa todo o sistema. Adicionalmente, a evolução tecnológica torna-se complexa, já que a atualização de tecnologias (como Java ou Spring Boot) exige a migração completa do sistema, aumentando o risco e o esforço associado.

Por fim, a arquitetura monolítica apresenta limitações na sua adequação a sistemas distribuídos modernos, não suportando naturalmente padrões como comunicação assíncrona ou observabilidade distribuída. Assim, a adoção de microsserviços surge como uma alternativa promissora, permitindo maior modularidade e flexibilidade, embora introduza novos desafios que justificam uma análise cuidadosa dos respetivos trade-offs.

Sistema de Referência

O sistema de referência utilizado neste projeto é o XCommerce Monolithic (Spring Boot), disponível publicamente e desenvolvido por Omar IRAQI. Trata-se de uma aplicação de e-commerce implementada em Java, baseada no framework Spring Boot, e estruturada segundo um modelo MVC (Model-View-Controller) com uma organização interna orientada a serviços, aproximando-se de um estilo SOA (Service-Oriented Architecture). Apesar dessa organização modular ao nível do código, toda a aplicação é executada

como um único artefacto monolítico, conforme definido no ficheiro de build (build.gradle).

A arquitetura do sistema integra múltiplas tecnologias amplamente utilizadas no desenvolvimento de aplicações empresariais, nomeadamente JPA/Hibernate para persistência de dados, PostgreSQL como base de dados relacional, Redis como mecanismo de cache, comunicação baseada em REST APIs, e um sistema de autenticação suportado por JSON Web Tokens (JWT). Todas estas componentes estão fortemente acopladas no mesmo processo de execução, partilhando a mesma infraestrutura e ciclo de deployment.

Do ponto de vista funcional, o domínio de negócio corresponde a uma plataforma de e-commerce generalista, suportando a comercialização de diferentes tipos de produtos, incluindo eletrónica (por exemplo, telemóveis de marcas como OPPO, Samsung e Apple), bicicletas (Cruiser, Sport, Touring, Dirt) e livros (Literature, Fiction, Science). Os produtos encontram-se organizados segundo uma estrutura hierárquica baseada em marcas (Brand) e categorias (Category), sendo estas últimas suportadas por relações hierárquicas através de um atributo parentCategory. O sistema inclui ainda funcionalidades típicas de e-commerce, como gestão de utilizadores, carrinho de compras, processamento de encomendas e gestão administrativa.

Este relatório considera o sistema XCommerce como o estado inicial (AS-IS), servindo de base para a análise arquitetural e para o processo de refactoring. A partir deste ponto de partida, foi realizada uma transformação para uma arquitetura baseada em microsserviços (TO-BE), cujo objetivo é permitir a avaliação dos impactos dessa transição em atributos de qualidade como performance, availability, deployability e custo.

Foco do Estudo

O presente relatório tem como objetivo analisar o sistema XCommerce no seu estado atual (AS-IS) e descrever a transformação arquitetural realizada para uma solução baseada em microsserviços (TO-BE), no contexto do projeto de Arquiteturas de Sistemas de Informação Distribuídos.

Numa primeira fase, é apresentada uma análise detalhada da arquitetura monolítica, incluindo a sua estrutura interna, principais componentes e tecnologias utilizadas. Esta análise permite compreender a organização do sistema, bem como as dependências existentes entre módulos funcionais.

De seguida, são identificados os principais pontos de acoplamento e dependências estruturais, evidenciando como diferentes partes do sistema interagem entre si e partilham recursos comuns, como a base de dados e o contexto de execução. Esta análise é fundamental para suportar decisões de decomposição arquitetural.

O relatório inclui ainda a análise de três workflows críticos do sistema, representados através de diagramas de sequência, com o objetivo de evidenciar dependências cruzadas entre componentes e identificar pontos sensíveis em termos de desempenho e isolamento de responsabilidades.

Com base nesta análise, é apresentada uma proposta de decomposição em microsserviços, acompanhada da respetiva justificação arquitetural. Esta proposta inclui a definição dos principais serviços, fronteiras funcionais e decisões relevantes ao nível da separação de responsabilidades e comunicação entre componentes.

Por fim, são discutidos os riscos associados à gestão de dados e consistência na transição para uma arquitetura distribuída, bem como apresentados resultados preliminares de testes operacionais, incluindo o comportamento do sistema perante falhas na base de dados. O relatório conclui com a formulação de hipóteses que irão orientar a avaliação experimental na fase final do projeto.

Arquitetura Base do Sistema Atual (AS-IS)

Visão Geral do Monólito

O XCommerce é uma plataforma de e-commerce desenvolvida como aplicação monolítica em Java, com o propósito de suportar operações típicas de comércio eletrônico: gestão de catálogo de produtos organizados por marcas e categorias hierárquicas, carrinho de compras com cálculo automático de preços e custos de envio, processamento de encomendas com máquina de estados, e gestão de utilizadores com autenticação baseada em JWT e autorização por roles.

A aplicação foi criada por Omar IRAQI como projeto de referência académico, seguindo uma arquitetura **MVC-SOA (Model-View-Controller com orientação a serviços)** num único artefacto executável Spring Boot.

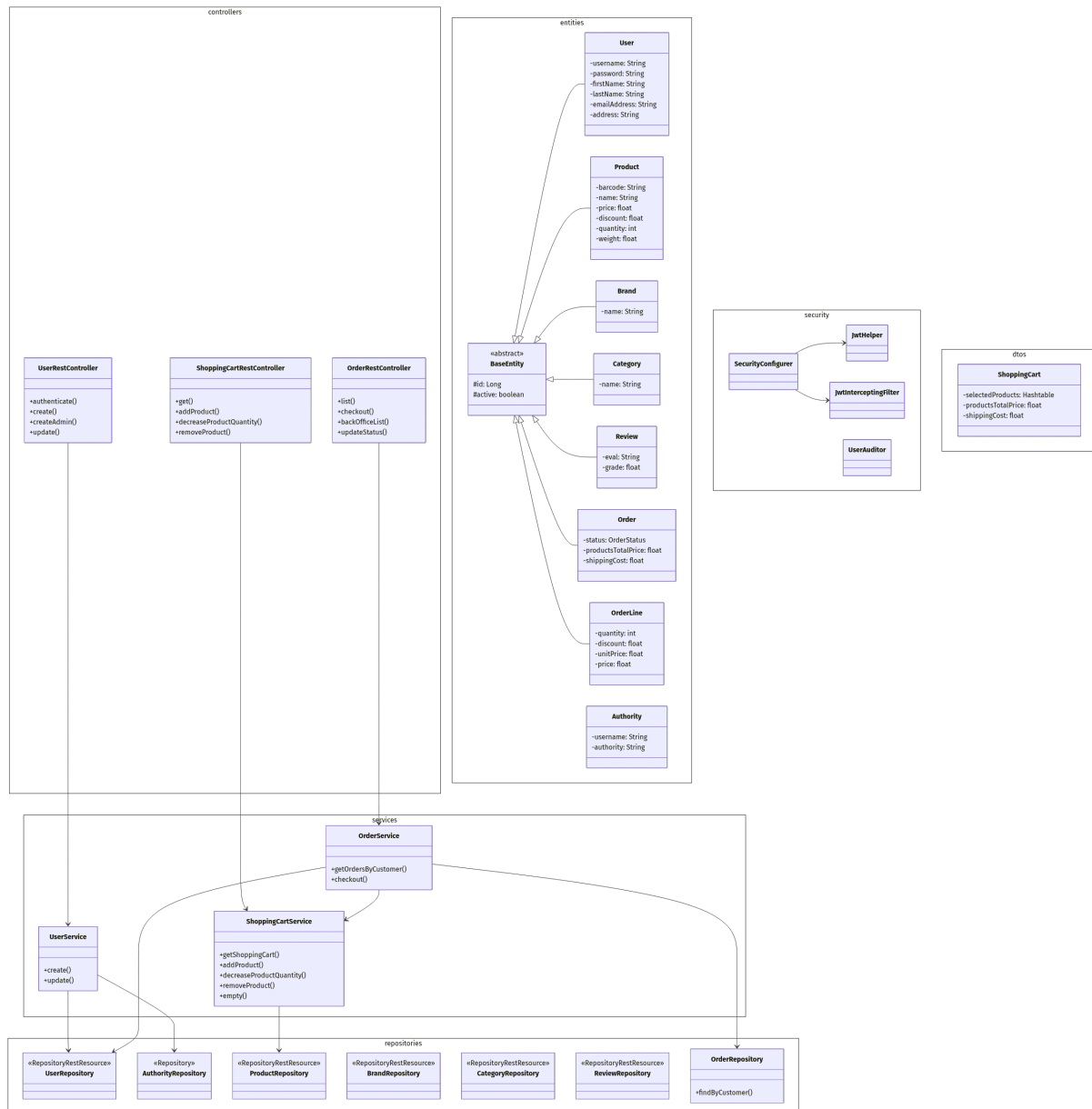
Stack Tecnológica

Componente	Tecnologia	Versão
Linguagem	Java	1.8 (Java 8)
Framework	Spring Boot	2.1.9.RELEASE
Gestão de dependências	Spring Dependency Management	1.0.8.RELEASE
Build tool	Gradle	Wrapper incluído
Persistência (ORM)	Spring Data JPA + Hibernate	Gerida pelo Spring Boot
Base de dados relacional	PostgreSQL	Driver incluído como <code>runtimeOnly</code>
Cache	Spring Data Redis	Via <code>spring-boot-starter-data-redis</code>
API REST (automática)	Spring Data REST	Via <code>spring-boot-starter-data-rest</code>

Componente	Tecnologia	Versão
API REST (manual)	Spring Web MVC	Via <code>spring-boot-starter-web</code>
Segurança	Spring Security + JWT	jjwt 0.10.7 + BCrypt
Monitorização	Spring Boot Actuator	Básico (health/info)

Organização por Camadas

O projeto contém **33 classes Java** no source principal (`src/main/java`), distribuídas em 5 packages:

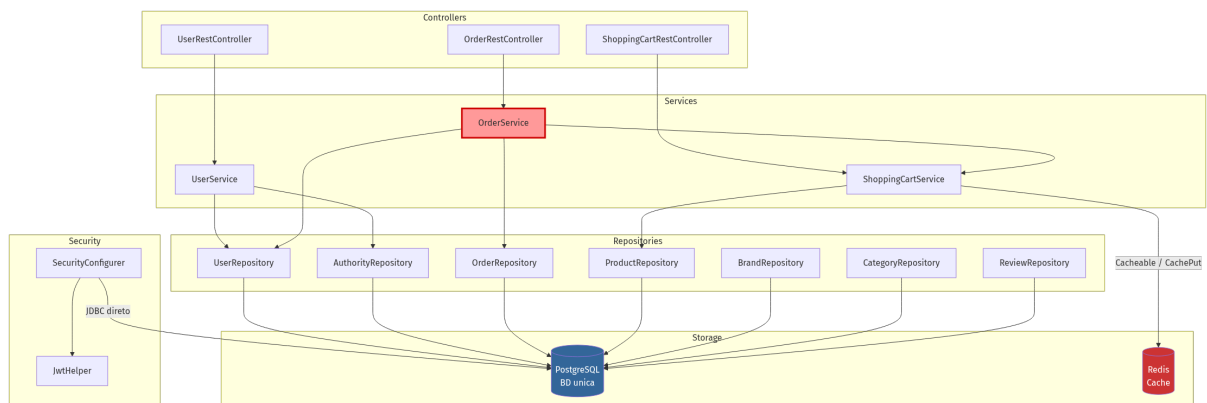


Package	Classes	Responsabilidade
monolithic (raiz)	1	XCommerceMonolithicApplication — ponto de entrada, ativa caching e auditing
monolithic.controllers	3	REST controllers manuais: UserRestController, OrderRestController, ShoppingCartRestController

Package	Classes	Responsabilidade
monolithic.services	3	Lógica de negócio: UserService, OrderService, ShoppingCartService
monolithic.data.entities	9	Entidades JPA: BaseEntity, User, Authority, Brand, Category, Product, Review, Order, OrderLine + enum OrderStatus + exceção IllegalStatusChangeException
monolithic.data.dtos	3	DTOs: User (DTO), Product (DTO), ShoppingCart
monolithic.data.repositories	7	Repositórios JPA: BaseRepository, UserRepository, AuthorityRepository, ProductRepository, BrandRepository, CategoryRepository, ReviewRepository, OrderRepository
monolithic.security	4	Segurança: SecurityConfigurer, JwtHelper, JwtInterceptingFilter, UserAuditor

Dependências Internas Principais

As dependências entre componentes formam um grafo onde os controllers dependem dos services, e os services dependem de múltiplos repositories, todos a aceder à mesma instância PostgreSQL:



OrderService é o componente mais acoplado: depende simultaneamente de `OrderRepository`, `UserRepository` e `ShoppingCartService`. O seu método `checkout()` cruza três domínios funcionais numa única operação.

ShoppingCartService depende de `ProductRepository` para obter dados atualizados de produtos e utiliza Redis como armazenamento via as anotações `@Cacheable` e `@CachePut`.

UserService depende de `UserRepository` e `AuthorityRepository` para gerir utilizadores e as suas roles de autorização.

SecurityConfigurer acede diretamente ao `DataSource` PostgreSQL via queries JDBC em texto livre para autenticação (`SELECT username, password, active FROM "user" WHERE username = ?`).

Não existe separação formal entre os boundaries de negócio, qualquer service pode injetar qualquer repository, e a segurança está diretamente acoplada às tabelas de domínio.

Principais Blocos Funcionais

A análise do código-fonte permite identificar **quatro blocos funcionais** coesos, cada um com responsabilidades distintas mas que partilham a mesma base de dados e o mesmo ciclo de deployment.

Bloco 1 — Autenticação e Gestão de Utilizadores

Aspeto	Detalhe
Responsabilidade	Registo de contas, autenticação por credenciais, geração/validação de tokens JWT, gestão de roles (SUPERADMIN, ADMIN, USER)
Classes	<code>UserRestController</code> , <code>UserService</code> , <code>SecurityConfigurer</code> , <code>JwtHelper</code> , <code>JwtInterceptingFilter</code> , <code>UserAuditor</code>

Aspeto	Detalhe
Entidades	User (tabela "user"), Authority (tabela authority)
Repositórios	UserRepository, AuthorityRepository
Endpoints	POST /rest/user/authenticate (login) · POST /rest/user (criar user) · POST /rest/user/admin (criar admin) · PATCH /rest/user/{id} (atualizar perfil)
Segurança	Autenticação JDBC direta contra a BD (usersByUsernameQuery, authoritiesByUsernameQuery). JWT HMAC-512 com validade de 24h. Passwords encriptadas com BCrypt.

A autenticação e a gestão de perfis de utilizador estão **completamente acopladas**: a mesma tabela "user" contém simultaneamente credenciais de acesso (username, password) e dados pessoais (firstName, lastName, emailAddress, address). Não há separação entre identidade e perfil.

Bloco 2 — Catálogo de Produtos

Aspeto	Detalhe
Responsabilidade	Gestão de produtos (CRUD), organização por marcas e categorias hierárquicas, pesquisa multi-critério, gestão de reviews
Classes	Sem controller manual dedicado — o catálogo é exposto automaticamente via Spring Data REST
Entidades	Product, Brand, Category, Review
Repositórios	ProductRepository (@RepositoryRestResource), BrandRepository (@RepositoryRestResource),

Aspeto	Detalhe
	<code>CategoryRepository</code> (<code>@RepositoryRestResource</code>), <code>ReviewRepository</code> (<code>@RepositoryRestResource</code>)
Endpoints (auto-gerados)	<code>GET/POST/PUT/DELETE /products · /brands · /categories · /reviews</code> (Spring Data REST expõe automaticamente CRUD + HATEOAS)
Queries custom	<code>ProductRepository</code> define 8 métodos JPQL para pesquisa por nome, categoria, marca, faixa de preço e combinações destes critérios

O catálogo é particularmente interessante porque não tem service layer explícito — toda a lógica de acesso é delegada diretamente ao Spring Data REST. Isto simplifica o código, mas significa que não há camada de validação de negócio entre o pedido HTTP e a persistência.

As categorias suportam **hierarquia** via auto-referência (`Category.parentCategory` → `Category`), com dados seed que incluem até 3 níveis: Root → Electronics → Phones/Workstations/Laptops e Root → Books → Literature/Fiction/Science.

Bloco 3 — Carrinho de Compras

Aspeto	Detalhe
Responsabilidade	Gestão do carrinho por utilizador: adicionar/remover produtos, ajustar quantidades, calcular preço total e custo de envio, esvaziar carrinho
Classes	<code>ShoppingCartRestController</code> , <code>ShoppingCartService</code> , DTO <code>ShoppingCart</code> , DTO <code>Product</code>
Entidades	Nenhuma — o carrinho não é uma entidade JPA. É um DTO (<code>ShoppingCart</code>) armazenado em Redis

Aspeto	Detalhe
Repositórios	<code>ProductRepository</code> (para obter dados atualizados de produtos)
Endpoints	<code>GET /rest/shoppingCart/get</code> · <code>PATCH /rest/shoppingCart/addProduct</code> · <code>PATCH /rest/shoppingCart/decreaseProductQuantity</code> · <code>PATCH /rest/shoppingCart/removeProduct</code>
Armazenamento	Redis via Spring Cache (<code>@Cacheable("ShoppingCart")</code> , <code>@CachePut("ShoppingCart")</code>)

Este bloco tem uma implementação particular. O `ShoppingCart` é um DTO serializável que contém um `Hashtable<Product, Integer>` — ou seja, a chave do mapa é a **entidade JPA completa** `Product` (com marca, categoria, reviews). Isto cria um acoplamento forte:

Qualquer alteração à classe `Product` pode invalidar os carrinhos já serializados no Redis.

O preço e o desconto são recalculados a cada operação, iterando sobre todos os produtos no carrinho. Isto é intencional (o comentário no código diz: *"We could just update them, but a discount may be applied by back office meanwhile"*), mas implica que o carrinho depende sempre do estado mais recente do `Product`.

O custo de envio é calculado com base no peso total dos produtos (`product.getWeight() * quantity`), com uma taxa base fixa de 25.

Nota: O controller tem o `userId` hardcoded (`long userId = 1; //To be removed`), com um comentário indicando que deveria ser extraído do JWT. A integração completa com o contexto de segurança não foi finalizada no monólito.

Bloco 4 — Encomendas

Aspeto	Detalhe
Responsabilidade	Checkout do carrinho, criação de encomendas com linhas de detalhe, consulta de encomendas por cliente, atualização de estado por back-office
Classes	<code>OrderRestController</code> , <code>OrderService</code>
Entidades	<code>Order</code> (tabela "order"), <code>OrderLine</code> , enum <code>OrderStatus</code> (HANDLING, SHIPPED, DELIVERED, ONHOLD, CANCELED), <code>IllegalStatusChangeException</code>
Repositórios	<code>OrderRepository</code>
Endpoints	GET <code>/rest/order/list</code> (encomendas do cliente) · GET <code>/rest/order/checkout</code> (finalizar) · GET <code>/rest/order/backOffice/list?customerId=X</code> · GET <code>/rest/order/backOffice/updateStatus?orderId=X&newStatus=Y</code>

O checkout é o processo mais complexo do monólito. O método `OrderService.checkout()`:

1. Obtém o `User` da BD
2. Obtém o `ShoppingCart` do Redis
3. Cria um `Order` a partir do carrinho (itera o `Hashtable<Product, Integer>` para criar `OrderLines`)
4. Persiste a `Order` com `CascadeType.ALL` (as `OrderLines` são persistidas em cascata)
5. Esvazia o carrinho no Redis

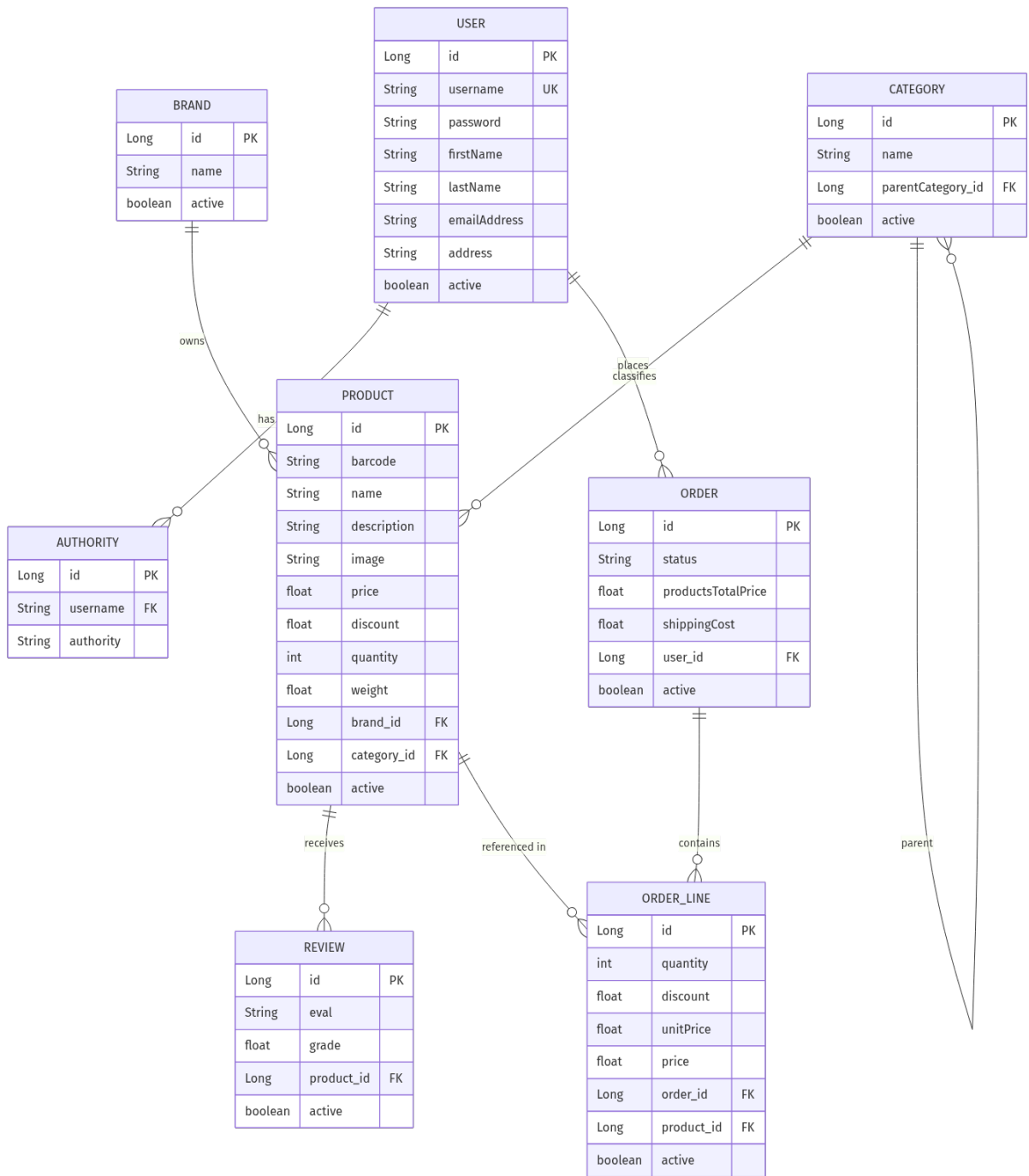
A `Order` tem uma máquina de estados simples: pode transitar de estados intermédios para qualquer novo estado, excepto a partir de CANCELED ou DELIVERED (que são terminais, lançando `IllegalStatusChangeException`).

Funcionalidades ausentes no monólito:

Funcionalidade	Estado
Gestão de inventário/stock	Não implementada — o campo <code>Product.quantity</code> existe mas nunca é validado nem decrementado no checkout
Processamento de pagamentos	Não implementado — a encomenda é criada diretamente com status HANDLING
Notificações	Não implementado — existe um comentário no <code>Order.updateStatus()</code> : <i>"use JavaMail API to send a notification to the customer by email"</i> , mas o código nunca foi escrito

Modelo de Dados AS-IS

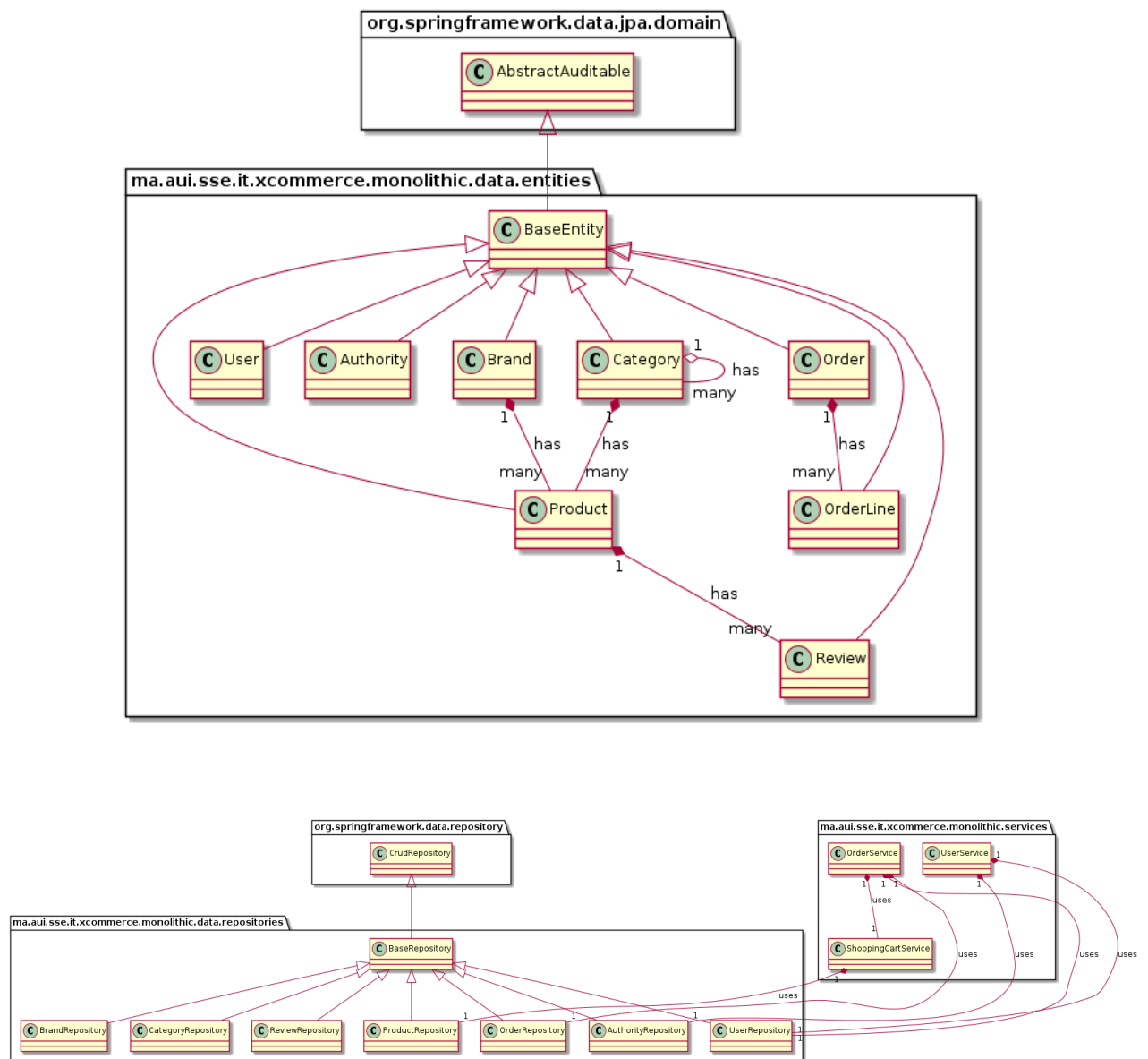
Toda a persistência do monólito reside numa **única base de dados PostgreSQL**, acedida por todos os componentes da aplicação. As entidades JPA mapeiam diretamente para tabelas relacionais com chaves estrangeiras que cruzam os boundaries funcionais.



Entidades e Relações

Entidade	Tabela	Campos-chave	Tipo
BaseEntity	(MappedSuperclass — não gera tabela)	id (Long, auto-generated), active (boolean)	Superclasse abstrata de todas as entidades
User	"user"	username (@NotNull, único), password (BCrypt hash), firstName, lastName, emailAddress, address	Credenciais + perfil
Authority	authority	username (@NonNull), authority (String — ex: "ROLE_SUPERADMIN")	Roles de autorização
Brand	brand	name (@NotNull)	Marca de produtos
Category	category	name (@NotNull), parentCategory (FK → Category, nullable)	Categoria hierárquica
Product	product	barcode (@NotNull), name, description, image, price (float), discount (float), quantity (int), weight (float)	Produto
Review	review	eval (String), grade (float)	Avaliação de produto
Order	"order"	status (enum OrderStatus), productsTotalPrice, shippingCost	Encomenda
OrderLine	order_line	quantity, discount, unitPrice, price (calculado)	Linha de detalhe

Relações entre Entidades



Dados Seed

A aplicação inclui um ficheiro `data.sql` que inicializa a base de dados com:

1 utilizador root (ROLE_SUPERADMIN)

3 marcas (OPPO, SAMSUNG, APPLE)

14 categorias em hierarquia de 3 níveis (Root → Electronics → Phones/Workstations/Laptops; Root → Bikes → Cruiser/Sport/Touring/Dirt; Root → Books → Literature/Fiction/Science)

4 produtos (F1 Plus, S10, iPhone X, iPhone 11 — todos na categoria Phones, com stock de 5 unidades cada)

Dependências Estruturais Relevantes

A análise das dependências entre módulos revela três padrões de acoplamento críticos que motivam a decomposição.

Mapa de Dependências Componente → Entidade

Componente (Service/Controller)	Entidades lidas	Entidades escritas	Repositories usados
UserRestController + UserService	User, Authority	User, Authority	UserRepository, AuthorityRepository
SecurityConfigurer	User (via SQL direto), Authority (via SQL direto)	—	— (usa DataSource JDBC)
ShoppingCartRestController + ShoppingCartService	Product	— (Redis only)	ProductRepository
OrderRestController + OrderService	User, ShoppingCart (Redis), Product (via ShoppingCart)	Order, OrderLine	OrderRepository, UserRepository
Spring Data REST (auto)	Product, Brand, Category, Review	Product, Brand, Category, Review	ProductRepository, BrandRepository, CategoryRepository, ReviewRepository

Dependência 1: Autenticação ↔ Domínio

A classe `SecurityConfigurer` configura a autenticação via JDBC diretamente contra as tabelas de domínio:

Isto significa que:

A autenticação não funciona se a BD de domínio estiver indisponível (SPOF)

Alterações ao schema da tabela `"user"` (renomear colunas, mudar tipo) quebram a autenticação

Não é possível escalar ou migrar o módulo de autenticação independentemente do resto da aplicação

A tabela `"user"` contém simultaneamente dados de segurança (`password`, `active`) e dados de perfil (`firstName`, `lastName`, `address`) — viola o princípio de single responsibility

Dependência 2: Carrinho ↔ Catálogo

O `ShoppingCart` é um DTO serializável que utiliza a **entidade JPA completa** `Product` como chave da sua estrutura de dados interna.

Na prática, quando o `ShoppingCartService.addProduct()` é invocado:

1. Faz `productRepository.findById(productId)` — vai à BD PostgreSQL buscar a entidade `Product` completa (com `Brand`, `Category`, lista de `Reviews`)
2. Coloca essa entidade como chave do `Hashtable` no DTO
3. Serializa o DTO inteiro e armazena no Redis via `@CachePut`

As consequências são:

O carrinho tem uma dependência forte no schema de `Product` — se a entidade mudar, os carrinhos em Redis podem falhar na desserialização

O método `equals()` e `hashCode()` do `Product` comparam por `id`, mas a entidade completa é serializada (incluindo campos que podem mudar, como `price` e `discount`)

Para recalcular preços, o carrinho itera sobre todos os produtos e lê `getPrice()` e `getDiscount()` dos objetos `Product` em memória — se o preço mudou na BD depois de adicionado ao carrinho, o preço no carrinho só é atualizado quando o utilizador adiciona ou remove outro produto

Dependência 3: Encomenda ↔ Todos os Domínios

O método `OrderService.checkout()` é o ponto de maior acoplamento de toda a aplicação.

Numa única transação, este método:

Lê a entidade `User` da BD (módulo Autenticação)

Lê o `ShoppingCart` do Redis (módulo Carrinho), que contém entidades `Product` (módulo Catálogo)

Escreve `Order` e `OrderLines` na BD (módulo Encomendas)

Escreve no Redis para esvaziar o carrinho (módulo Carrinho)

Se qualquer destes passos falhar, a transação JPA faz rollback da persistência na BD, mas o estado do Redis pode ficar inconsistente (ou vice-versa, dado que Redis e PostgreSQL não partilham a mesma transação).

Adicionalmente, o construtor `new Order(shoppingCart, user)` itera toda a `Hashtable<Product, Integer>` do carrinho para criar `OrderLines`, acedendo a `product.getPrice()`, `product.getDiscount()` e `product.getWeight()`. Isto significa que os valores gravados na `OrderLine` dependem do estado do `Product` no momento da criação — se o preço mudar entre o momento em que o produto foi adicionado ao carrinho e o momento do checkout, o valor gravado será o da última leitura cachada, não necessariamente o valor atual na BD.

Síntese das Dependências Cruzadas

Conclusão da análise AS-IS: O monólito XCommerce apresenta três níveis de acoplamento que dificultam a evolução, escalabilidade e resiliência do sistema:

1. **Acoplamento de dados** — todas as entidades partilham a mesma BD PostgreSQL, com FKs cruzadas entre domínios (`User ↔ Order ↔ Product`)
2. **Acoplamento de código** — o `OrderService` depende explicitamente de components de 3 domínios diferentes; o `ShoppingCart` contém entidades JPA completas do catálogo
3. **Acoplamento operacional** — uma falha na BD PostgreSQL torna toda a aplicação inoperacional (SPOF); não é possível escalar, deployar ou monitorizar módulos individuais

Workflows Analisados

Critérios de Escolha dos Workflows

Para analisar o impacto da decomposição arquitetural, foram selecionados três workflows que, em conjunto, cobrem o ciclo completo de uma experiência de compra na plataforma XCommerce. A escolha seguiu quatro critérios:

Representatividade funcional. Os três workflows cobrem as operações core de qualquer plataforma de e-commerce: navegação/interação com o catálogo (adicionar ao carrinho), consulta de dados históricos (listar encomendas) e a operação transacional mais crítica (checkout). Juntos, representam a maioria do tráfego funcional que uma aplicação deste tipo recebe.

Uso de múltiplos blocos funcionais. Cada workflow envolve pelo menos dois dos quatro blocos identificados na Secção 2.2. O Workflow 1 cruza Carrinho e Catálogo; o Workflow 2 cruza Encomendas, Utilizadores e Catálogo; o Workflow 3 cruza todos os quatro blocos (Autenticação/Utilizadores, Catálogo, Carrinho e Encomendas). Esta característica é essencial para expor as dependências cruzadas que a decomposição deve resolver.

Relevância arquitetural. Os workflows selecionados exercitam diretamente os três padrões de acoplamento identificados na Secção 2.4: a dependência entre autenticação e domínio (Workflow 2 e 3), o acoplamento entre carrinho e catálogo (Workflow 1 e 3) e a transação que cruza todos os módulos (Workflow 3).

Exposição de dependências críticas. Cada workflow permite analisar consequências concretas da decomposição: o Workflow 1 expõe a necessidade de sincronização de dados de produto entre serviços; o Workflow 2 expõe o problema de composição de leitura (API Composition); o Workflow 3 expõe os riscos de consistência quando uma transação atômica local é substituída por comunicação distribuída.

Workflow 1 — Adicionar Produto ao Carrinho

Descrição

Aspeto	Detalhe
Ator	Cliente autenticado
Pré-condição	O produto existe na base de dados e está ativo

Aspetto	Detalhe
Operação	Adicionar uma quantidade de um produto ao carrinho do utilizador
Resultado esperado	O carrinho é atualizado com o novo item (ou a quantidade é incrementada se o produto já estava no carrinho). Os totais de preço e envio são recalculados.
Blocos funcionais envolvidos	Carrinho + Catálogo

Fluxo no Monólito (passo a passo)

O cliente envia um pedido `PATCH /rest/shoppingCart/addProduct` com o body `{"id": <productId>, "quantity": <qty>}`. O processamento segue esta sequência:

Passo 1: Receção no Controller.

O `ShoppingCartRestController.addProduct()` recebe o pedido. Deveria extrair o `userId` do contexto de segurança JWT (como indica o comentário no código: `//Retrieve userId from JWT-based security context`), mas na implementação atual o valor está hardcoded como `userId = 1`.

Passo 2: Leitura do carrinho (Redis).

O controller invoca `shoppingCartService.getShoppingCart(userId)`. Este método está anotado com `@Cacheable(value = "ShoppingCart")`, o que significa que o Spring verifica primeiro o Redis. Se o carrinho já existe em cache, é retornado diretamente sem aceder à base de dados. Se não existe, é criado um novo `ShoppingCart` vazio e armazenado no Redis.

Passo 3 — Leitura do produto (PostgreSQL).

O controller invoca `shoppingCartService.addProduct(shoppingCart, userId, productId, quantity)`. Dentro deste método, a primeira operação é `productRepository.findById(productId).get()` — uma consulta à base de dados PostgreSQL para obter a entidade JPA `Product` completa, incluindo os seus campos `price`, `discount`, `weight`, e as relações `@ManyToOne` com `Brand` e `Category`.

Passo 4 — Atualização da estrutura de dados.

O método `ShoppingCart.addProduct(product, quantity)` delega para `updateProductQuantity(product, quantity)`, que:

- Verifica se o produto já está no `Hashtable<Product, Integer>` (usando `Product.equals()` que compara por `id`)
- Se já existe, soma a quantidade; se não existe, insere a nova entrada
- Se a quantidade resultante for zero, remove o produto do mapa
-

Passo 5 — Recálculo dos totais.

Ainda dentro de `updateProductQuantity()`, o método itera **todos os produtos** no `Hashtable` para recalcular os totais:

O preço total é calculado como: `preço × (1 - desconto/100) × quantidade` para cada produto. O custo de envio é calculado como: `peso × quantidade` para cada produto, mais uma taxa base de 25 (aplicada apenas se o carrinho não estiver vazio).

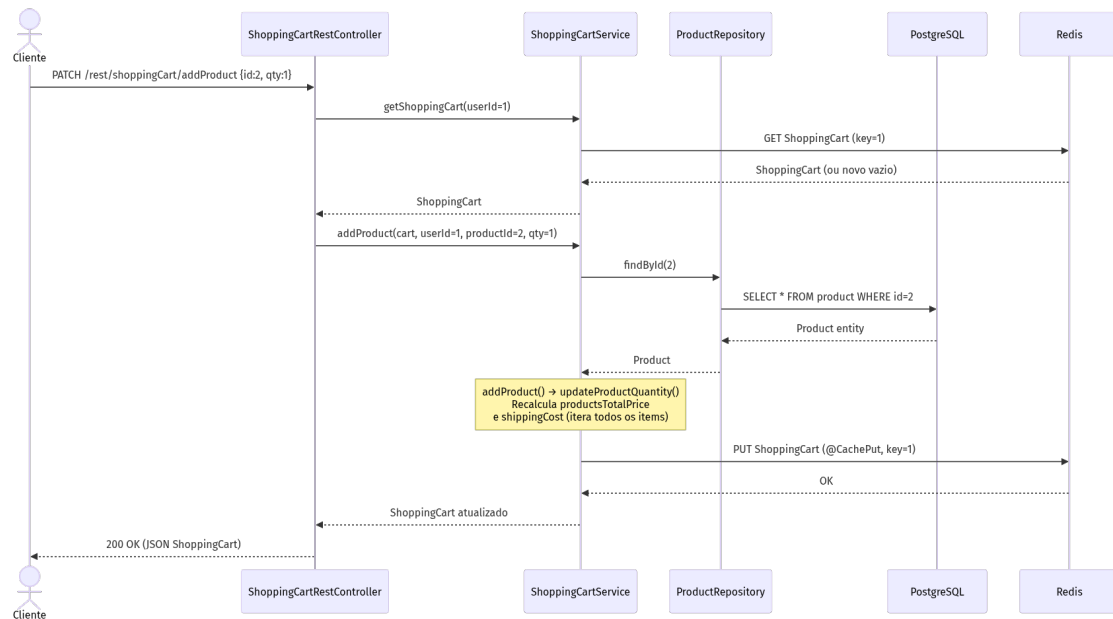
Passo 6 — Persistência no Redis.

O método `addProduct()` está anotado com `@CachePut(value = "ShoppingCart", key = "#userId")`. Ao retornar, o Spring Cache serializa o objeto `ShoppingCart` inteiro (incluindo o `Hashtable<Product, Integer>` com todas as entidades JPA `Product`) e grava-o no Redis, substituindo a versão anterior.

Passo 7 — Resposta ao cliente.

O controller retorna o `ShoppingCart` atualizado como JSON ao cliente.

Diagrama de Sequência



Dependências Cruzadas Identificadas

Dependência	Origem	Destino	Tipo	Impacto
Leitura de produto	ShoppingCartService	ProductRepository → PostgreSQL	Síncrona, obrigatória	Se a BD falhar, não é possível adicionar produtos ao carrinho, mesmo que o carrinho esteja em Redis
Serialização de entidade JPA	ShoppingCart (Redis)	Classe Product (JPA entity)	Estrutural	Alterações à classe Product podem invalidar carrinhos serializados em Redis
Cálculo de preço	ShoppingCart.updateProductQuantity()	Product.getPrice(), Product.getDiscount()	Dados em memória	Os preços usados no cálculo são os do momento da adição ao carrinho, não necessariamente os atuais na BD
Cálculo de envio	ShoppingCart.updateProductQuantity()	Product.getWeight()	Dados em memória	O peso é lido da entidade JPA em memória

Implicação para a decomposição: Na arquitetura de microserviços, o Cart Service e o Catalog Service são serviços separados com bases de dados independentes. O carrinho não pode conter entidades JPA do catálogo. Na versão TO-BE, o Cart Service armazena apenas `productId`, `quantity` e `username` — valores escalares sem dependência do schema do catálogo. Isto elimina o acoplamento estrutural, mas transfere a responsabilidade de resolução de preços para o momento do checkout (o Order Service consulta o Catalog Service para obter o preço atualizado).

Workflow 2 — Consultar Encomendas de um Cliente

Descrição

Aspeto	Detalhe
Ator	Cliente autenticado (as suas encomendas) ou Back-office staff (encomendas de qualquer cliente)
Pré-condição	O utilizador existe e tem encomendas registadas
Operação	Obter a lista de encomendas de um cliente, incluindo os detalhes de cada linha (produto, quantidade, preço)
Resultado esperado	Lista de objetos Order com OrderLines associadas
Blocos funcionais envolvidos	Encomendas + Utilizadores + Catálogo

Fluxo no Monólito (passo a passo)

Existem dois pontos de entrada para este workflow:

- **Cliente:** `GET /rest/order/list` — o `customerId` deveria ser extraído do JWT, mas está hardcoded como 1
- **Back-office:** `GET /rest/order/backOffice/list?customerId=X` — o `customerId` é passado como query parameter

Ambos convergem para a mesma operação interna:

Passo 1 — Invocação do serviço.\

O `OrderRestController` invoca
`orderService.getOrdersByCustomer(customerId).`

Passo 2 — Query JPQL.

O `OrderService` delega para

`orderRepository.findByCustomerId(customerId)`, que executa a query.

Esta é uma query JPQL que o Hibernate traduz para SQL nativo. O ponto importante é que `o.user.id` implica um **JOIN implícito** com a tabela `"user"` — o Hibernate precisa de verificar a relação `@ManyToOne` entre `Order` e `User` para resolver o filtro.

Passo 3 — Carregamento LAZY das OrderLines.

A estratégia `FetchType.LAZY` significa que as `OrderLines` não são carregadas pela query inicial. Só são carregadas quando o código (ou a serialização JSON) acede à propriedade `order.getOrderLines()`.

Passo 4 — Carregamento dos Products (via OrderLine).

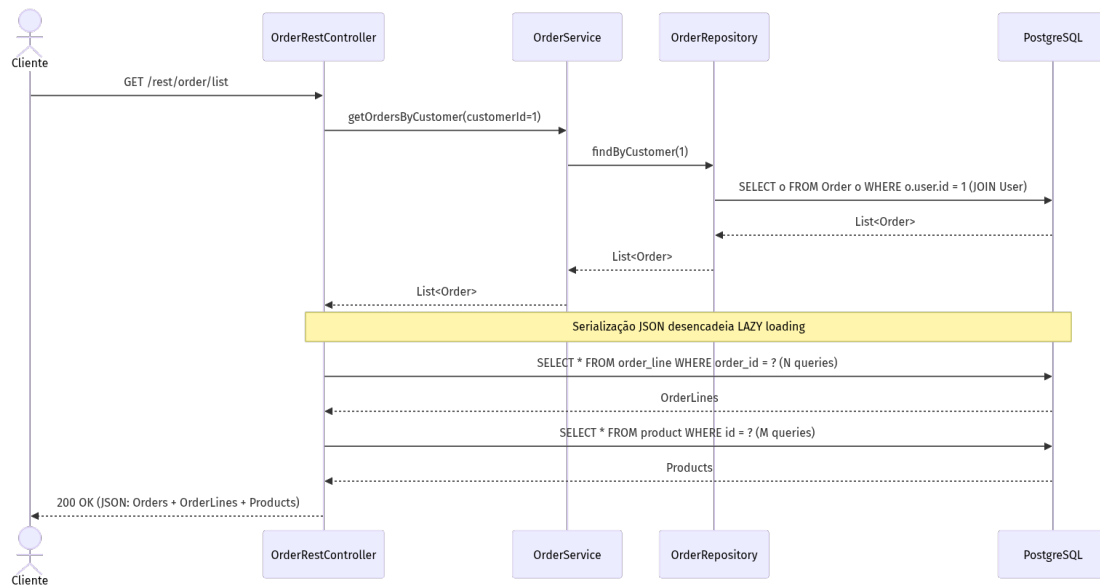
Cada `OrderLine` tem uma relação `@ManyToOne` com `Product`. Quando a serialização JSON acede a `orderLine.getProduct()`, o Hibernate executa uma query adicional para cada produto referenciado:

Isto pode resultar no problema clássico **N+1 queries**: 1 query para as Orders + N queries para as OrderLines + M queries para os Products de cada OrderLine.

Passo 5 — Resposta ao cliente.

O controller retorna a lista de `Orders` como JSON, com todos os dados hierárquicos resolvidos: cada Order contém as suas OrderLines, cada OrderLine contém o Product referenciado (com Brand e Category).

Diagrama de Sequência



Dependências Cruzadas Identificadas

Dependência	Origem	Destino	Tipo	Impacto
Filtro por utilizador	OrderRepository (JPQL)	Tabela "user" (JOIN implícito)	Query local	No monólito é transparente; nos microserviços, Order Service não tem acesso à tabela User
LAZY loading de OrderLines	Serialização JSON	Tabela order_line (N queries)	Query local	Transparente no monólito; nos microserviços, Order Service gere as suas próprias OrderItems
LAZY loading de Products	Serialização JSON (via OrderLine)	Tabela product (M queries)	Query local	No monólito, o Product completo é retornado; nos microserviços, o Order Service guarda apenas productId e price localmente (desnormalização)

Problema de composição de dados. Este workflow demonstra com clareza o custo da decomposição para operações de leitura. No monólito, uma única query JPQL (com LAZY loading transparente) retorna uma estrutura hierárquica completa — Orders com OrderLines, cada uma com dados completos de Product, Brand e Category. Tudo é resolvido dentro do mesmo processo JVM, contra a mesma base de dados.

Na versão microserviços, o Order Service guarda apenas dados escalares: `username` (String, em vez de FK para User), `productId` e `price` (copiados no momento da criação, em vez de FK para Product). Se o cliente precisar de informação adicional sobre o produto (nome, imagem, marca, categoria), seria necessária uma composição de dados — chamadas adicionais ao Catalog Service para enriquecer a resposta. Na implementação atual, o Order Service retorna dados desnormalizados (o preço gravado no momento da encomenda), evitando a composição mas sacrificando a atualidade dos dados de produto.

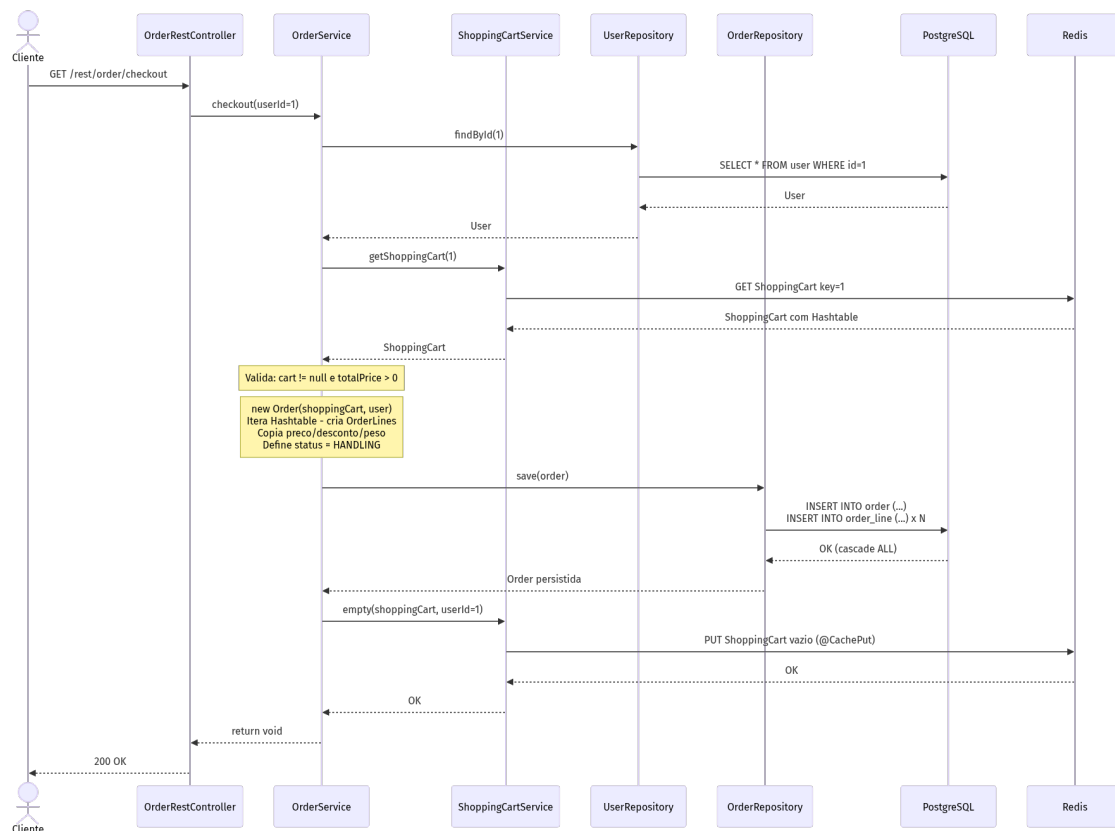
Workflow 3 — Finalizar Encomenda (Checkout)

Descrição

Aspeto	Detalhe
Ator	Cliente autenticado
Pré-condição	O cliente tem produtos no carrinho
Operação	Criar uma encomenda a partir do conteúdo atual do carrinho
Resultado esperado	Uma nova Order é criada com OrderLines correspondentes aos itens do carrinho; o carrinho é esvaziado
Blocos funcionais envolvidos	Todos os quatro: Utilizadores + Carrinho + Catálogo + Encomendas

Este é o workflow mais crítico da aplicação, tanto em termos de complexidade funcional como de impacto na decomposição arquitetural. É a única operação que cruza todos os blocos funcionais numa única transação.

Fluxo no Monólito (passo a passo)



O cliente envia `GET /rest/order/checkout`. O processamento é inteiramente sequencial:

Passo 1 — Receção no Controller.

O `OrderRestController.checkout()` obtém o `customerId` (hardcoded como 1) e invoca `orderService.checkout(customerId)`.

Passo 2 — Leitura do utilizador (PostgreSQL).

O `OrderService` executa `userRepository.findById(userId).get()`, que faz uma query à tabela "user".

Isto retorna a entidade `User` completa (username, password, firstName, lastName, emailAddress, address). O `User` é necessário porque a `Order` tem uma relação `@ManyToOne @NotNull` com `User`.

Passo 3 — Leitura do carrinho (Redis).

O `OrderService` invoca `shoppingCartService.getShoppingCart(userId)`, que lê o DTO `ShoppingCart` do cache Redis. Este objeto contém o `Hashtable<Product, Integer>` com todos os produtos previamente adicionados.

Passo 4 — Validação.

O serviço verifica se o carrinho é nulo ou está vazio (`productsTotalPrice == 0`). Se sim, retorna sem ação. **Nota:** não é verificado se os produtos ainda existem na BD, se estão ativos, ou se há stock suficiente (`Product.quantity`).

Passo 5 — Criação da Order e OrderLines.

O construtor `new Order(shoppingCart, user)`:

- Associa a `Order` ao `User` (FK direta)
- Itera o `Hashtable<Product, Integer>` do carrinho
- Para cada entrada, cria um `new OrderLine(this, product, quantity)` que:
 - Referencia a `Order` e o `Product` (ambas FKs)
 - Copia `product.getDiscount()` para `OrderLine.discount`
 - Copia `product.getPrice()` para `OrderLine.unitPrice`
 - Calcula `price = unitPrice × (1 - discount/100) × quantity`
- Copia o `productsTotalPrice` e o `shippingCost` do `ShoppingCart` para os campos da `Order`
- Define o estado como `OrderStatus.HANDLING`

Passo 6 — Persistência (PostgreSQL).

O `OrderService` executa `orderRepository.save(order)`. Como a relação `Order → OrderLine` está anotada com `CascadeType.ALL`, o Hibernate persiste a `Order` e todas as suas `OrderLines` numa única operação transacional

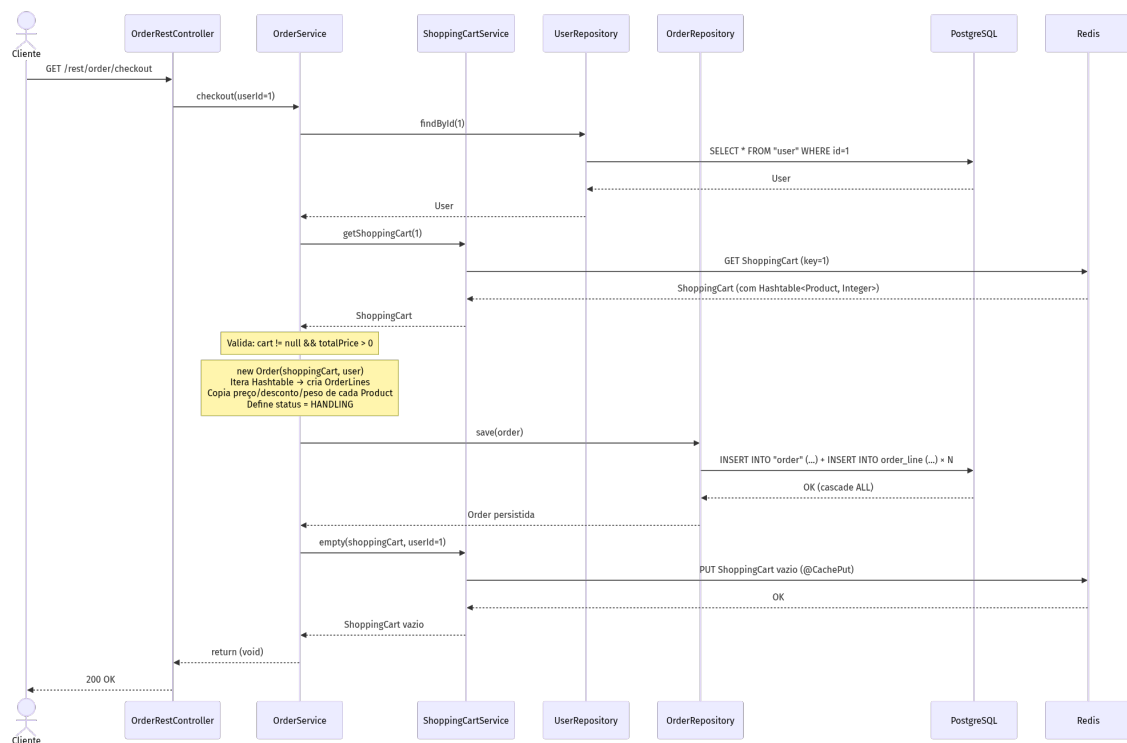
Passo 7 — Limpeza do carrinho (Redis).

O `OrderService` invoca `shoppingCartService.empty(shoppingCart,`

`user.getId()`), que reinicializa o `ShoppingCart` (novo `Hashtable`, totais a zero) e grava-o no Redis via `@CachePut`.

Garantia transacional: Os passos 5 e 6 estão dentro da mesma transação JPA (delimitada pelo Spring `@Transactional` implícito ao nível do service). Se o `orderRepository.save()` falhar, é feito rollback. **Contudo**, o passo 7 (limpeza do Redis) está fora do scope transacional JPA — se a persistência na BD falhar após o Redis ter sido limpo, o carrinho fica perdido. Na prática, o passo 7 é executado após o save, por isso este risco é mitigado, mas a separação de concerns entre PostgreSQL e Redis não é atômica.

Diagrama de Sequência



Problemas de Consistência Evidenciados

A análise do código do checkout revela quatro lacunas funcionais que, no monólito, são limitações de implementação, mas que na decomposição se tornam decisões arquiteturais explícitas:

1. Stock não é validado.

O monólito não verifica se `Product.quantity >= quantidade pedida` antes de criar a `Order`. A entidade `Product` tem o campo `quantity` (definido a 5 para os produtos seed), mas este valor nunca é consultado no fluxo de checkout. É possível criar encomendas para quantidades arbitrárias, mesmo que excedam o stock.

2. Stock não é decrementado.

Após a criação da encomenda, o campo `Product.quantity` não é atualizado. Não existe nenhuma operação do tipo `product.setQuantity(product.getQuantity() - orderedQuantity)`. O stock permanece inalterado independentemente do número de encomendas criadas.

3. Não há processamento de pagamento.

A encomenda é criada diretamente com o estado `OrderStatus.HANDLING`. Não existe validação de método de pagamento, nem processamento, nem confirmação. O comentário no código (`// use JavaMail API to send a notification`) sugere que funcionalidades adicionais eram planeadas mas nunca implementadas.

4. Não há notificação ao cliente.

O checkout retorna `void` — o cliente não recebe confirmação com número de encomenda, nem notificação por email ou outro canal. A única evidência de sucesso é o status HTTP 200.

Comparação com a Versão Microserviços (TO-BE)

Na arquitetura de microserviços, o checkout é substancialmente diferente. O `OrderService` do `order-service` colmata as lacunas do monólito, mas introduz complexidade distribuída:

Aspeto	Monólito (AS-IS)	Microserviços (TO-BE)
Identificação do utilizador	<code>userRepository.findById(userId)</code> — FK para entidade User	<code>username (String)</code> extraído do header <code>X-User-Name</code> injetado pelo Gateway
Leitura do carrinho	<code>shoppingCartService.getShoppingCart()</code> — chamada local in-process ao Redis	<code>orderRemoteService.getCartItems(username)</code> — chamada HTTP via Feign ao Cart Service (porta 8083)
Validação de stock	Não existe	<code>orderRemoteService.checkStock(productId, quantity)</code> — chamada HTTP via Feign ao Inventory Service (porta 8085)
Obtenção de preço	<code>product.getPrice()</code> — lido da entidade JPA em memória (cacheada no ShoppingCart)	<code>orderRemoteService.getProductPrice(productId)</code> — chamada HTTP via Feign ao Catalog Service (porta 8082)
Decremento de stock	Não existe	<code>orderRemoteService.decreaseStock(productId, quantity)</code> — chamada HTTP via Feign ao Inventory Service
Persistência	<code>orderRepository.save()</code> — transação local JPA com <code>CascadeType.ALL</code>	<code>repository.save()</code> — transação local na BD do Order Service (<code>xcommerce_orders</code>)
Limpeza do carrinho	<code>shoppingCartService.empty()</code> — chamada local + <code>@CachePut</code> Redis	<code>orderRemoteService.clearCart(username)</code> — chamada HTTP DELETE via Feign ao Cart Service
Pagamento	Não existe	Evento Kafka <code>order-placed-events</code> consumido pelo Payment Service (processamento assíncrono, ~80% sucesso simulado)

Aspeto	Monólito (AS-IS)	Microserviços (TO-BE)
Notificação	Não existe	Evento Kafka <code>order-placed-events</code> consumido pelo Notification Service (log de confirmação)
Resiliência	Transação atômica (rollback JPA)	Resilience4j: Circuit Breaker (failureRate 50%, slidingWindow 8), Retry (3 tentativas, 500ms), Bulkhead (20 chamadas concorrentes)
Número de operações de rede	0 (tudo local)	4+ chamadas HTTP síncronas + 1 publish Kafka

Síntese das Dependências Observadas

Aspeto	WF1 — Adicionar ao Carrinho	WF2 — Consultar Encomendas	WF3 — Checkout
Blocos funcionais	Carrinho + Catálogo	Encomendas + Utilizadores + Catálogo	Todos (User + Cart + Catálogo + Encomendas)
Entidades tocadas	Product (leitura), ShoppingCart DTO (Redis)	Order, OrderLine, User, Product	User, ShoppingCart, Product, Order, OrderLine
Tipo de operação	Leitura BD + Escrita cache	Leitura com JOINS implícitos (LAZY)	Leitura múltipla + Escrita BD + Limpeza cache

Aspeto	WF1 — Adicionar ao Carrinho	WF2 — Consultar Encomendas	WF3 — Checkout
Nº de acessos à BD	1 (SELECT product)	1 + N + M (N+1 queries problem)	2 (SELECT user, INSERT order + order_lines)
Acessos ao Redis	2 (GET + PUT)	0	2 (GET + PUT)
Transação atômica	Não (cache e BD são independentes)	Sim (leitura pura)	Parcial (BD é transacional, Redis não)
Dependência cruzada principal	Cart depende de Product (preço/peso)	Order depende de User (FK) e Product (via OrderLine)	Transação que cruza todos os módulos
Risco na decomposição	Sincronização de dados de produto	Composição de leitura (API Composition)	Consistência distribuída (sem ACID global)

Dependências Comuns

A análise dos três workflows revela padrões recorrentes:

1. Product é o ponto de acoplamento central.

A entidade `Product` aparece em todos os três workflows: é lida para adicionar ao carrinho (WF1), é referenciada pelas OrderLines nas consultas de encomendas (WF2), e os seus dados de preço, desconto e peso são copiados para as OrderLines no checkout (WF3). Na versão microserviços, esta dependência é transformada de FK relacional para referência por `productId` (inteiro), com desnormalização de dados no momento da escrita.

2. User é partilhado entre autenticação e encomendas.

No monólito, a mesma tabela "user" serve para autenticação JDBC (Secção 2.4, Dependência 1) e como FK na tabela "order". O checkout precisa do User como entidade JPA para satisfazer a relação @ManyToOne @NotNull. Na versão microserviços, o Order Service usa apenas a string username (propagada via header HTTP pelo Gateway), eliminando a dependência da entidade User.

3. O checkout é a operação de maior risco.

O Workflow 3 é o único que faz **escrita em múltiplos stores** (PostgreSQL + Redis) e **leitura de todos os domínios** numa única operação. No monólito, a atomicidade é parcialmente garantida pela transação JPA (mas não cobre o Redis). Na versão microserviços, a atomicidade é substituída por:

- Chamadas síncronas via Feign (com Resilience4j para falhas transitórias)
- Eventos assíncronos via Kafka (para pagamento e notificação)
- Aceitação de eventual consistency em vez de strong consistency

4. O padrão de comunicação muda radicalmente.

Workflow	Monólito: Nº operações de rede	Microserviços: Nº operações de rede
WF1 — Adicionar ao carrinho	0 (in-process)	0 (Cart Service opera localmente)
WF2 — Consultar encomendas	0 (in-process)	0 (Order Service retorna dados locais desnormalizados)
WF3 — Checkout	0 (in-process)	4+ chamadas HTTP + 1 evento Kafka

O checkout é o workflow cujo custo de comunicação mais aumenta. No monólito, todo o fluxo (ler user, ler carrinho, criar order, limpar carrinho) acontece dentro do mesmo processo JVM sem qualquer operação de rede (excepto acessos a PostgreSQL e Redis, que estão na mesma máquina). Na versão microserviços, o Order Service executa pelo

menos: 1 chamada ao Cart Service (ler items), N chamadas ao Inventory Service (verificar stock $\times$ N produtos), N chamadas ao Catalog Service (obter preço $\times$ N produtos), N chamadas ao Inventory Service (decrementar stock $\times$ N produtos), 1 chamada ao Cart Service (limpar carrinho) e 1 publicação Kafka. Para um carrinho com 3 produtos, isto representa aproximadamente 11 operações de rede.

Os workflows analisados demonstram que a decomposição em microserviços:

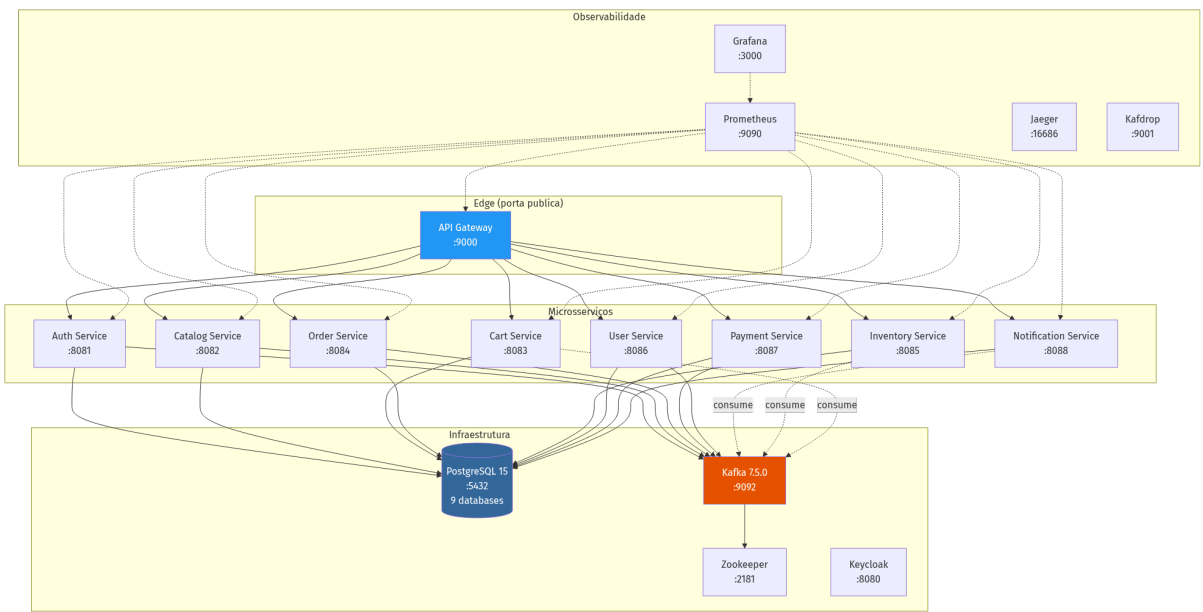
- **Resolve** o acoplamento estrutural (FKs cruzadas, entidades JPA partilhadas, BD única)
- **Resolve** o SPOF na base de dados (cada serviço tem a sua BD)
- **Resolve** as lacunas funcionais do monólito (o checkout nos microserviços valida stock, decrementa inventário, processa pagamento e notifica)
- **Introduz** latência adicional no checkout (comunicação inter-serviço)
- **Introduz** risco de inconsistência (falha parcial entre chamadas remotas)
- **Exige** mecanismos de resiliência (Circuit Breaker, Retry, Bulkhead) que não existiam no monólito
- **Transfere** a complexidade de JOINS relacionais para composição de APIs ou desnormalização explícita

Estas implicações serão quantificadas na avaliação experimental (Secção 7) e validadas contra as hipóteses formuladas na Secção 5.

Arquitetura TO-BE (Microserviços)

Variante TO-BE Adotada

A arquitetura-alvo decompõe o monólito XCommerce em **8 microserviços de domínio** mais um **API Gateway**, deployados como 16 containers Docker via Docker Compose. Cada serviço é uma aplicação Spring Boot 3.4.0 autónoma, com a sua própria base de dados PostgreSQL, expondo uma REST API e comunicando por HTTP síncrono (OpenFeign) ou mensagens assíncronas (Apache Kafka).



Aspeto	Monólito (AS-IS)	Microserviços (TO-BE)
Linguagem	Java 8	Java 21
Framework	Spring Boot 2.1.9.RELEASE	Spring Boot 3.4.0
Cloud framework	—	Spring Cloud 2024.0.0
Build tool	Gradle	Maven
Base de dados	PostgreSQL (1 BD única, ~10 tabelas)	PostgreSQL 15 (9 BDs isoladas)

Aspeto	Monólito (AS-IS)	Microserviços (TO-BE)
Cache	Redis (ShoppingCart como @Cacheable)	— (cart em BD relacional)
Segurança	Spring Security + JWT manual (HMAC-512, jjwt 0.10.7)	JWT via auth0/java-jwt 4.4.0 + Keycloak (IAM externo)
Comunicação inter-módulo	Chamada de método Java (in-process)	REST síncrono (OpenFeign) + Kafka assíncrono
Resiliência	—	Resilience4j (Circuit Breaker + Retry + Bulkhead)
Observabilidade	—	Prometheus + Grafana (métricas), Jaeger + OpenTelemetry (tracing), Kafdrop (Kafka UI)
API Docs	Spring Data REST (auto-exposed)	SpringDoc OpenAPI 2.8.5 (Swagger UI por serviço, agregado no Gateway)
Deploy	JAR único	Docker Compose (16 containers)
Gestão de schema	data.sql (seed)	Flyway (migrações versionadas por serviço)

Composição do Deployment

O `docker-compose.yml` define 16 containers organizados em 4 camadas:

Camada	Container	Porta	Função
Infraestrutura	<code>xcommerce-db</code> (PostgreSQL 15)	5432	BD partilhada (9 databases lógicas)

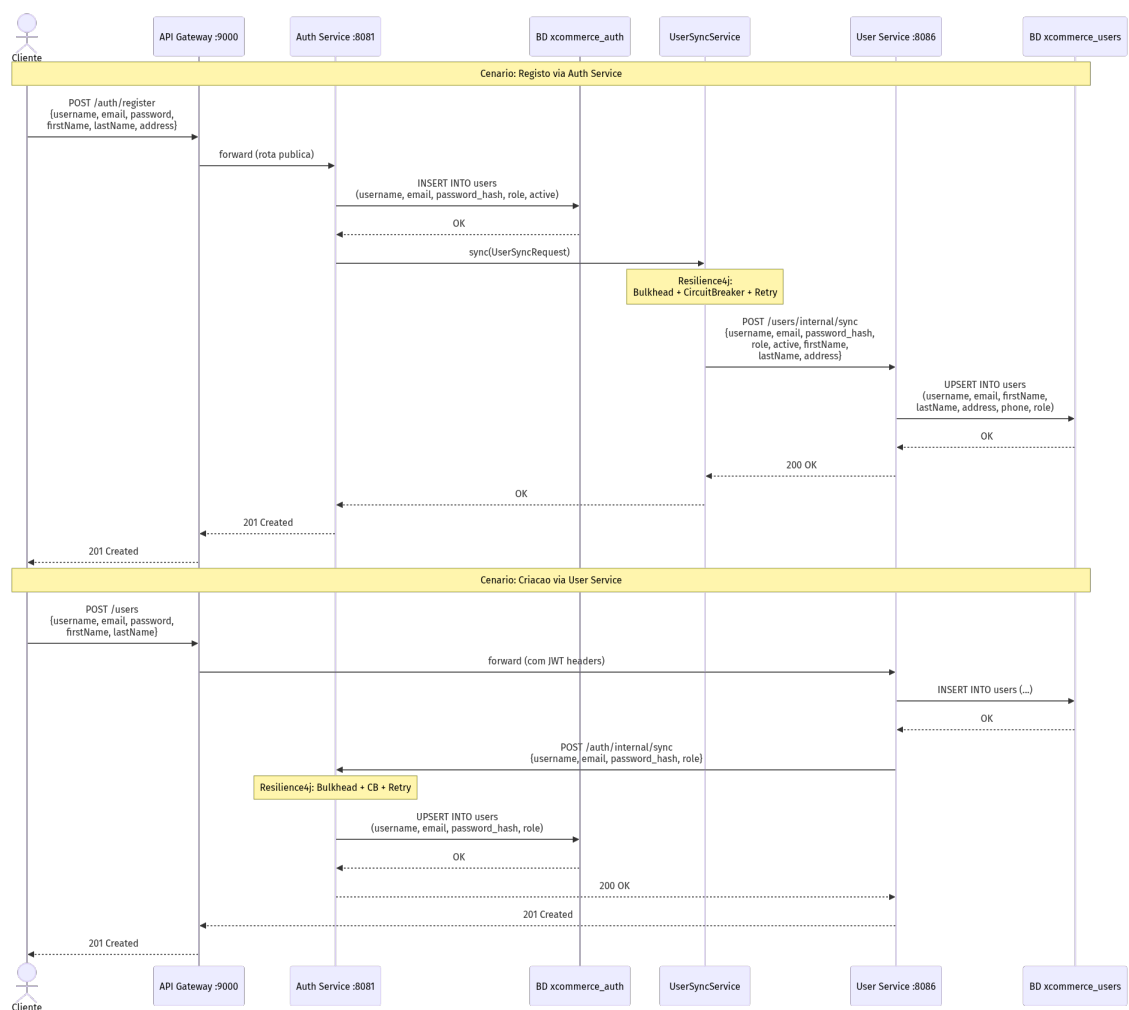
Camada	Container	Porta	Função
	xcommerce-zookeeper	2181	Coordenação Kafka
	xcommerce-kafka	9092	Message broker
	xcommerce-keycloak	8080	IAM / OAuth2 (preparado, não obrigatório)
Observabilidade	xcommerce-kafdrop	9001	UI Kafka
	xcommerce-jaeger	16686	Distributed tracing
	xcommerce-prometheus	9090	Métricas
	xcommerce-grafana	3000	Dashboards
Serviços	xcommerce-auth-service	8081	Autenticação
	xcommerce-catalog-service	8082	Catálogo
	xcommerce-cart-service	8083	Carrinho
	xcommerce-order-service	8084	Encomendas
	xcommerce-inventory-service	8085	Inventário
	xcommerce-user-service	8086	Perfis de utilizador
	xcommerce-payment-service	8087	Pagamentos

Camada	Container	Porta	Função
	xcommerce-notification-service	8088	Notificações
Edge	xcommerce-gateway	9000	API Gateway

Justificação das Fronteiras

Porquê 8 serviços (e não mais, nem menos)?

A decomposição seguiu cinco princípios:



1. Coesão funcional — bounded contexts do domínio.

Cada serviço encapsula um bounded context distinto de e-commerce:

Serviço	Bounded Context	Responsabilidade
Auth Service	Identidade e Acesso	Registo, login, emissão e validação JWT
Catalog Service	Catálogo de Produtos	CRUD de produtos, marcas, categorias e reviews
Cart Service	Carrinho de Compras	Gestão de itens do carrinho por utilizador
Order Service	Gestão de Encomendas	Criação, listagem e atualização de estado de encomendas
Inventory Service	Inventário / Stock	Controlo de quantidades disponíveis
User Service	Perfis de Utilizador	Dados pessoais (nome, morada, telefone)
Payment Service	Pagamentos	Processamento (simulado) de pagamentos
Notification Service	Notificações	Fan-out de alertas (login, produto, encomenda, pagamento)

Nenhum serviço contém lógica que pertence a dois bounded contexts. O Catalog Service não sabe o que é uma encomenda; o Cart Service não sabe o que é um preço.

2. Autonomia de dados — Database per Service.

O Docker Compose cria 9 databases lógicas numa única instância PostgreSQL.

Cada serviço liga-se exclusivamente à sua database. O Catalog Service não consegue fazer `SELECT` na tabela de orders, porque a tabela simplesmente não existe na database `xcommerce_catalog`. Esta separação é lógica (mesmo servidor PostgreSQL), mas pode ser facilmente convertida em separação física (servidores distintos) sem alterar código.

3. Redução de acoplamento — substituição de dependências.

Dependência AS-IS	Solução TO-BE
Auth + User na mesma tabela "user"	Auth Service tem tabela <code>users</code> (credenciais). User Service tem tabela <code>users</code> (perfil). Sync bidirecional via REST <code>/internal/sync</code> .
ShoppingCart serializa entidades JPA <code>Product</code> no Redis	Cart Service armazena apenas (<code>username</code> , <code>productId</code> , <code>quantity</code>) em tabela relacional. Sem Redis, sem entidades JPA do catálogo.
Checkout acede <code>User</code> , <code>ShoppingCart</code> , <code>Product</code> , <code>Order</code> numa transação	Order Service orquestra via Feign (<code>Cart</code> → <code>Inventory</code> → <code>Catalog</code> → <code>Inventory</code> → <code>persist</code> → <code>Cart clear</code>) com Resilience4j.

4. Alinhamento com workflows.

O Workflow 3 (checkout), identificado na Secção 3 como a maior zona de acoplamento, é agora orquestrado explicitamente pelo Order Service. Cada dependência que era implícita (chamada de método in-process) torna-se uma chamada HTTP explícita com fallback, retry e circuit breaker.

5. Separação Auth vs User — split do bounded context de identidade.

No monólito, a tabela "user" continha tanto credenciais (`username`, `password`, `role`) como dados de perfil (`firstName`, `lastName`, `address`). Na decomposição:

Serviço	Tabela	Campos	BD
Auth Service	<code>users</code>	<code>id</code> , <code>username</code> , <code>email</code> , <code>password (hash)</code> , <code>role</code> , <code>active</code>	<code>xcommerce_auth</code>

Serviço	Tabela	Campos	BD
User Service	users	id, username, email, password (hash), first_name, last_name, address, phone, role, active	xcommerce_users

A sincronização funciona assim:

- **No registo (Auth → User):** O `AuthController.register()` persiste as credenciais localmente e chama `UserSyncService.sync()`, que faz POST `http://user-service:8086/users/internal/sync` com todos os dados (incluindo `firstName`, `lastName`, `address` do `RegisterRequest`). A chamada usa `Resilience4j` (Bulkhead + CircuitBreaker + Retry).
- **Na criação via User Service (User → Auth):** O `UserController.createUser()` persiste o perfil localmente e chama `AuthSyncService`, que faz POST `http://auth-service:8081/auth/internal/sync` com `username`, `email`, `password hash` e `role`.
- **Endpoint interno:** Ambos os serviços expõem POST `/internal/sync` que faz upsert (insert ou update) baseado em `username/email`.

Papel do API Gateway

O API Gateway (Spring Cloud Gateway, porta 9000) é o único ponto de entrada para clientes externos. Os 8 serviços internos não expõem portas no Docker Compose (comunicam apenas via rede interna `xcommerce-network`).

Routing

O Gateway define **10 rotas de negócio** e **8 rotas de documentação**, configuradas programaticamente no `ApiGatewayApplication.gatewayRoutes()`:

Rota	Path Predicate	Serviço Destino	URI	Circuit Breaker
auth-service	/auth/**	Auth Service	http://auth-service:8081	auth-gateway-cb → /fallback/auth

Rota	Path Predicate	Serviço Destino	URI	Circuit Breaker
catalog-service	/products/**	Catalog Service	http://catalog-service:8082	catalog-gateway-cb → /fallback/catalog
brand-service	/brands/**	Catalog Service	http://catalog-service:8082	catalog-brand-gateway-cb → /fallback/catalog
category-service	/categories/**	Catalog Service	http://catalog-service:8082	catalog-category-gateway-cb → /fallback/catalog
cart-service	/cart/**	Cart Service	http://cart-service:8083	cart-gateway-cb → /fallback/cart
order-service	/order/*	Order Service	http://order-service:8084	order-gateway-cb → /fallback/order
inventory-service	/inventory/**	Inventory Service	http://inventory-service:8085	inventory-gateway-cb → /fallback/inventory
user-service	/users/*	User Service	http://user-service:8086	user-gateway-cb → /fallback/users
payment-service	/payments/**	Payment Service	http://payment-service:8087	payment-gateway-cb → /fallback/payments

Rota	Path Predicate	Serviço Destino	URI	Circuit Breaker
notification-service	/notifications/*	Notification Service	http://notification-service:8088	notification-gateway-cb → /fallback/notifications

As 8 rotas de documentação mapeiam `/docs/{servico}` para `/v3/api-docs` do serviço respetivo (ex: `/docs/catalog` → `http://catalog-service:8082/v3/api-docs`), e o Swagger UI agregado no Gateway lista todas as APIs num único interface.

APIs Externas (por Serviço)

Auth Service (porta 8081) — BD: `xcommerce_auth`

Método	Endpoint	Descrição	Auth
POST	/auth/register	Registo de novo utilizador. Cria credenciais locais + sync REST ao User Service.	Público
POST	/auth/login	Login. Valida password (BCrypt), emite JWT (HMAC256, issuer <code>auth-api</code>). Publica Kafka <code>user-events</code> .	Público
POST	/auth/authenticate	Alias de <code>/auth/login</code> (compatibilidade).	Público
POST	/auth/internal/sync	Upsert de credenciais (chamado pelo User Service).	Interno

Comunica com:

- **User Service** (REST síncrono): POST `http://user-service:8086/users/internal/sync` — no registo, sincroniza perfil
- **Kafka produces**: topic `user-events` (evento de login detetado)

Catalog Service (porta 8082) — BD: `xcommerce_catalog`

Método	Endpoint	Descrição	Auth
GET	<code>/products</code>	Lista todos os produtos (com DTO: id, name, price, discount, brand, category, image...)	Público
GET	<code>/products/search?name=&brand=&category=&minPrice=&maxPrice=</code>	Pesquisa filtrada (in-memory stream filter)	Público
GET	<code>/products/{id}</code>	Detalhe de um produto	Público
POST	<code>/products</code>	Criar produto. Sync stock → Inventory Service. Publica Kafka <code>product-events</code> .	Admin
PUT	<code>/products/{id}</code>	Atualizar produto. Re-sync stock + Kafka event.	Admin
DELETE	<code>/products/{id}</code>	Eliminar produto	Admin
GET	<code>/products/{id}/price</code>	Retorna apenas o preço (BigDecimal). Usado pelo Order Service via Feign.	Protegido
GET	<code>/products/{id}/reviews</code>	Lista reviews do produto	Público

Método	Endpoint	Descrição	Auth
POST	/products/{id}/reviews	Criar review	Protegido
GET	/brands	Lista todas as marcas	Público
POST	/brands	Criar marca	Admin
PUT	/brands/{id}	Atualizar marca	Admin
PATCH	/brands/{id}/deactivate	Desativar marca	Admin
GET	/categories	Lista todas as categorias	Público
POST	/categories	Criar categoria	Admin
PUT	/categories/{id}	Atualizar categoria	Admin
PATCH	/categories/{id}/deactivate	Desativar categoria	Admin

Comunica com:

- **Inventory Service** (REST síncrono): POST `http://inventory-service:8085/inventory/sync?productId=&quantity=` — sync stock ao criar/atualizar produto (via `InventorySyncService` com `Resilience4j`)
- **Kafka produces**: topic `product-events` (payload: `"productId:quantity"`)

Cart Service (porta 8083) — BD: `xcommerce_cart`

Método	Endpoint	Descrição	Auth
GET	/cart	Lista items do carrinho (username via header <code>X-User-Name</code>)	Protegido

Método	Endpoint	Descrição	Auth
GET	/cart/get	Alias de GET /cart	Protegido
POST	/cart	Adicionar item (productId + quantity). Se já existe, incrementa.	Protegido
POST	/cart/add	Alias de POST /cart	Protegido
PATCH	/cart/addProduct	Alias de POST /cart (compatibilidade com monólito)	Protegido
PATCH	/cart/decreaseProductQuantity	Decrementar quantidade. Se chega a 0, remove.	Protegido
PATCH	/cart/removeProduct	Remover item do carrinho	Protegido
DELETE	/cart	Limpar todo o carrinho do utilizador	Protegido

Comunica com:

- **Nenhum serviço** (operação totalmente local). Armazena apenas (username, productId, quantity).
- **Kafka consomes:** topic user-events (groupId cart-group) — quando deteta login, limpa o carrinho do utilizador (comportamento de sessão).

Nota: O Cart Service é o exemplo mais "puro" de autonomia — opera sem qualquer dependência de outros serviços. Não precisa do preço nem do nome do produto.

Order Service (porta 8084) — BD: xcommerce_orders

Método	Endpoint	Descrição	Auth
POST	/order	Criar encomenda direta (sem carrinho)	Protegido
POST	/order/checkout	Checkout: orquestra Cart → Inventory → Catalog → persist → Kafka	Protegido
GET	/order/checkout	Alias GET do checkout (compatibilidade com monólito)	Protegido
GET	/order/{id}	Detalhe de encomenda (owner ou admin)	Protegido
GET	/order/list	Encomendas do utilizador (por X-User-Name)	Protegido
GET	/order/backOffice/list	Encomendas (admin). Aceita ?username= ou ?customerId=	Admin
PATCH	/order/{id}/status	Atualizar estado da encomenda	Admin
GET	/order/backOffice/updateStatus	Alias de atualização de estado (compatibilidade)	Admin

Comunica com (REST síncrono via Feign + Resilience4j):

- **Cart Service:** GET /cart (obter items) + DELETE /cart (limpar após checkout)
- **Inventory Service:** GET /inventory/check (validar stock) + POST /inventory/decrease (reservar)
- **Catalog Service:** GET /products/{id}/price (obter preço atualizado)

Kafka produces: topic order-placed (username) + topic order-placed-events (payload "productId:quantity")

Kafka consumes: topic `payment-events` (groupId `order-group`) — via `OrderPaymentListener`:

- `PAYMENT_SUCCESSFUL` → publica `order-confirmed-events`
- `PAYMENT_FAILED` → publica `order-cancelled-events`

O Order Service é o **serviço orquestrador** — é o único que faz chamadas Feign a três outros serviços. Cada chamada é decorada com `Bulkhead` → `CircuitBreaker` → `Retry` via `OrderRemoteService`.

Inventory Service (porta 8085) — BD: `xcommerce_inventory`

Método	Endpoint	Descrição	Auth
GET	<code>/inventory/check?productId=&quantity=</code>	Verifica se há stock suficiente (retorna boolean)	Interno/Protegido
POST	<code>/inventory/decrease?productId=&quantity=</code>	Decrementa stock	Interno/Protegido
POST	<code>/inventory/sync?productId=&quantity=</code>	Sync/upsert stock (chamado pelo Catalog Service)	Interno

Comunica com:

- **Nenhum serviço** (operação local)
- **Kafka consumes:**
 - topic `product-events` (groupId `inventory-group`) — ao detetar novo produto no catálogo, cria/atualiza o registo de stock
 - topic `order-placed-events` (groupId `inventory-group`) — ao detetar encomenda, decrementa stock automaticamente

User Service (porta 8086) — BD: `xcommerce_users`

Método	Endpoint	Descrição	Auth
POST	/users	Criar utilizador (CUSTOMER). Sync → Auth Service. Publica Kafka <code>user-events</code> .	Público/Protegido
POST	/users/admin	Criar admin. Exige role ADMIN.	Admin
POST	/users/super-admin	Criar super-admin. Exige role SUPERADMIN.	SuperAdmin
GET	/users	Listar todos os utilizadores	Admin
GET	/users/backOffice/list	Alias de listagem (compatibilidade)	Admin
GET	/users/{id}	Obter utilizador por ID (owner ou admin)	Protegido
GET	/users/by-username/{username}	Obter por username	Protegido
PATCH	/users/{id}	Atualizar perfil	Protegido
POST	/users/internal/sync	Upsert de perfil (chamado pelo Auth Service no registo)	Interno

Comunica com:

- **Auth Service** (REST síncrono): POST `http://auth-service:8081/auth/internal/sync` — no criar utilizador, sincroniza credenciais
- **Kafka produces**: topic `user-events` (payload: `"USER_CREATED"`)

Objetivos, Questões de Investigação e Hipóteses

Objetivos do Estudo

O presente estudo avalia a refatorização do monólito XCommerce numa arquitetura de 8 microserviços + API Gateway, com cinco objetivos:

O1. Avaliar se a decomposição reduz o acoplamento estrutural entre módulos funcionais — medido pela eliminação de foreign keys cruzadas, entidades JPA partilhadas e transações que atravessam múltiplos bounded contexts.

O2. Avaliar se a arquitetura de microserviços melhora o isolamento de falhas — verificando se a indisponibilidade de um serviço (ex: Payment Service) impede ou não o funcionamento dos restantes (ex: navegação no catálogo, gestão de carrinho).

O3. Analisar o impacto da decomposição na complexidade de operações de leitura composta e checkout — comparando o número de operações de rede, a necessidade de API Composition e os mecanismos de resiliência exigidos.

O4. Avaliar os riscos introduzidos na consistência de dados num sistema distribuído — nomeadamente em stock (Inventory), encomendas (Order) e carrinho (Cart), quando transações ACID locais são substituídas por comunicação REST síncrona e eventos Kafka assíncronos.

O5. Comparar ambas as arquiteturas em quatro atributos de qualidade: **Performance** (latência de checkout), **Availability** (tolerância a falhas parciais), **Deployability** (independência de deploy por serviço) e **Cost** (overhead operacional de 16 containers vs 1 JAR).

Questões de Investigação

ID	Questão
QI1	A decomposição proposta melhora efetivamente a separação de responsabilidades e a independência de deploy?
QI2	Que workflows ficam mais simples e quais ficam mais complexos após a transição para microserviços?
QI3	Que impactos surgem na consistência dos dados quando as transações atômicas do monólito são substituídas por comunicação inter-serviço (REST + Kafka)?
QI4	O API Gateway consegue suportar eficazmente os cenários de routing, autenticação e fallback sem introduzir gargalos de performance?
QI5	Qual é o overhead de latência introduzido pela comunicação inter-serviço comparativamente às chamadas in-process do monólito?

Mapeamento QI → Objetivos

QI	Objetivos relacionados	Secções que respondem
QI1	O1, O5 (Deployability)	Secção 4.2 (fronteiras), Secção 7 (métricas de acoplamento)
QI2	O3	Secção 3 (workflows AS-IS), Secção 6 (workflows TO-BE)
QI3	O4	Secção 3.4 (checkout AS-IS), Secção 4.5 (Kafka saga), Secção 7 (cenários de falha)
QI4	O2, O5 (Availability, Performance)	Secção 4.3 (Gateway), Secção 7 (testes de fallback)
QI5	O3, O5 (Performance)	Secção 7 (benchmarks de latência)

Hipóteses

ID	Hipótese	Variável independente	Variável dependente	Critério de validação
H1	A decomposição melhora a modularidade e o isolamento de falhas — a indisponibilidade do Payment Service não afeta o Catalog browsing nem o Cart.	Arquitetura (monólito vs microserviços)	Disponibilidade funcional parcial	Derrubar um serviço e verificar se os restantes continuam operacionais (fallback 503 apenas na rota afetada).
H2	Workflows de leitura agregada (ex: consultar encomenda com detalhes de produto) tornam-se mais complexos, exigindo API Composition ou desnormalização.	Decomposição da BD	Número de chamadas de rede para resolver uma leitura	Comparar: monólito (0 chamadas, LAZY loading local) vs microserviços (N chamadas Feign ou dados desnormalizados).
H3	A separação da BD única em 9 bases independentes aumenta o risco de inconsistências temporárias (ex: stock decrementado no Inventory mas encomenda não gravada no Order	Database per Service	Janela de inconsistência	Simular falha entre chamadas Feign no checkout e verificar estado de stock vs encomenda.

ID	Hipótese	Variável independente	Variável dependente	Critério de validação
	Service em caso de falha parcial).			
H4	O API Gateway com Circuit Breaker reduz a complexidade exposta ao cliente e permite degradação graciosa.	Presença de Gateway + CB	Resposta ao cliente quando um serviço está down	Derrubar serviço downstream e verificar: cliente recebe 503 com mensagem amigável (não timeout nem erro genérico).
H5	O Resilience4j (Circuit Breaker + Retry + Bulkhead) mitiga falhas transitórias entre serviços, mas não elimina o risco de inconsistência em falhas prolongadas.	Configuração Resilience 4j	Taxa de sucesso do checkout sob falhas intermitentes vs prolongadas	Injetar latência/falhas no Inventory Service: com retries, checkout recupera; com serviço down >30s, circuit breaker abre e checkout falha.

Mapeamento Hipóteses → QI

Hipótese	QI que responde
H1	QI1, QI4
H2	QI2, QI5
H3	QI3
H4	QI4

Hipótese	QI que responde
H5	QI3, QI5

Dados, Consistência e Migração

Tabela de Ownership dos Dados

Serviço	BD	Tabelas principais	Entidade-chave	Dados que possui
Auth Service	xcommerce_auth	users	User	username, email, password (BCrypt hash), role, active
Catalog Service	xcommerce_catalog	products, brands, categories, reviews	Product, Brand, Category, Review	Nome, preço, desconto, peso, barcode, imagem, marca, categoria, reviews
Cart Service	xcommerce_cart	cart_items	CartItem	username, productId, quantity
Order Service	xcommerce_orders	orders, order_items	Order, OrderItem	username, status, productId, quantity, price (copiado), shippingCost
Inventory Service	xcommerce_inventory	inventory	Inventory	productId, quantity
User Service	xcommerce_users	users	User	username, email, password (hash), first_name, last_name,

Serviço	BD	Tabelas principais	Entidade-chave	Dados que possui
				address, phone, role, active
Payment Service	xcommerce_payment	payments	Payment	orderId, amount, status
Notification Service	xcommerce_notification	— (apenas logging)	—	Não persiste dados; consome eventos Kafka e faz log

Comparação com o Monólito

No monólito, todas as entidades residiam na mesma BD PostgreSQL:

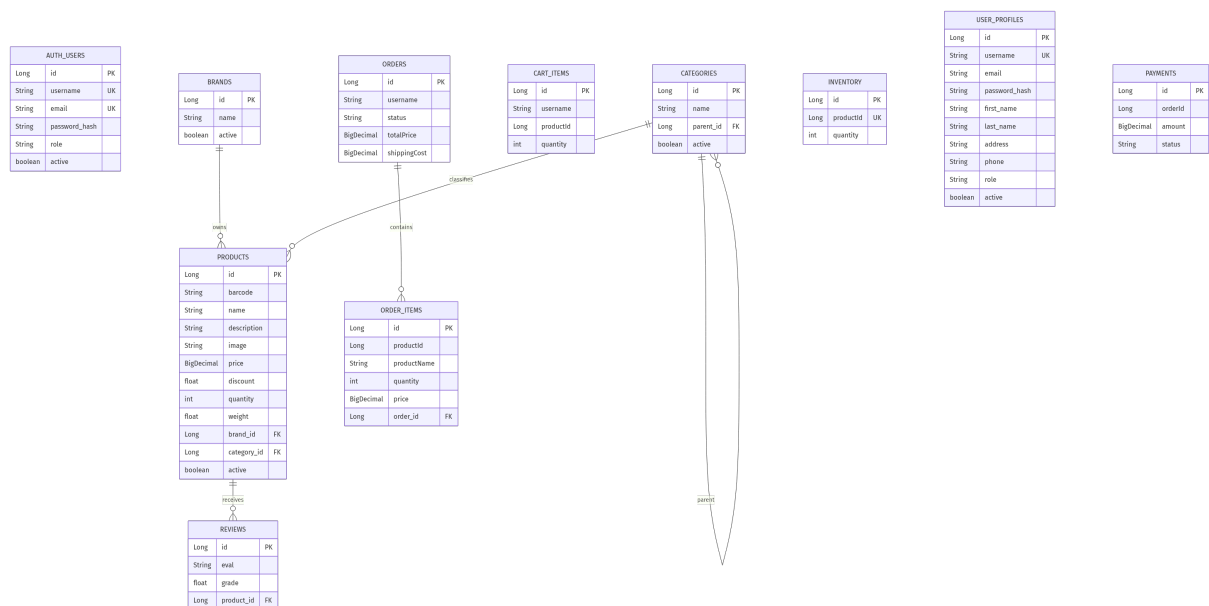


Tabela monólito	Serviço TO-BE	Transformação
"user" (credenciais + perfil)	Auth Service + User Service	Split em duas BDs com sync REST bidirecional

Tabela monólito	Serviço TO-BE	Transformação
product	Catalog Service	Migração direta; campo quantity replicado no Inventory
brand	Catalog Service	Migração direta
category	Catalog Service	Migração direta
"order"	Order Service	FK user_id substituída por username (String)
order_line	Order Service (como order_items)	FK product_id mantida como productId (Long, sem FK real); price desnormalizado
ShoppingCart (Redis DTO)	Cart Service (tabela relacional)	Hashtable<Product, Integer> substituído por CartItem(username, productId, quantity)
— (não existia)	Inventory Service	Serviço novo; stock extraído de Product.quantity
— (não existia)	Payment Service	Serviço novo
— (não existia)	Notification Service	Serviço novo

6.2 Dependências de Dados

JOINS que deixaram de ser locais

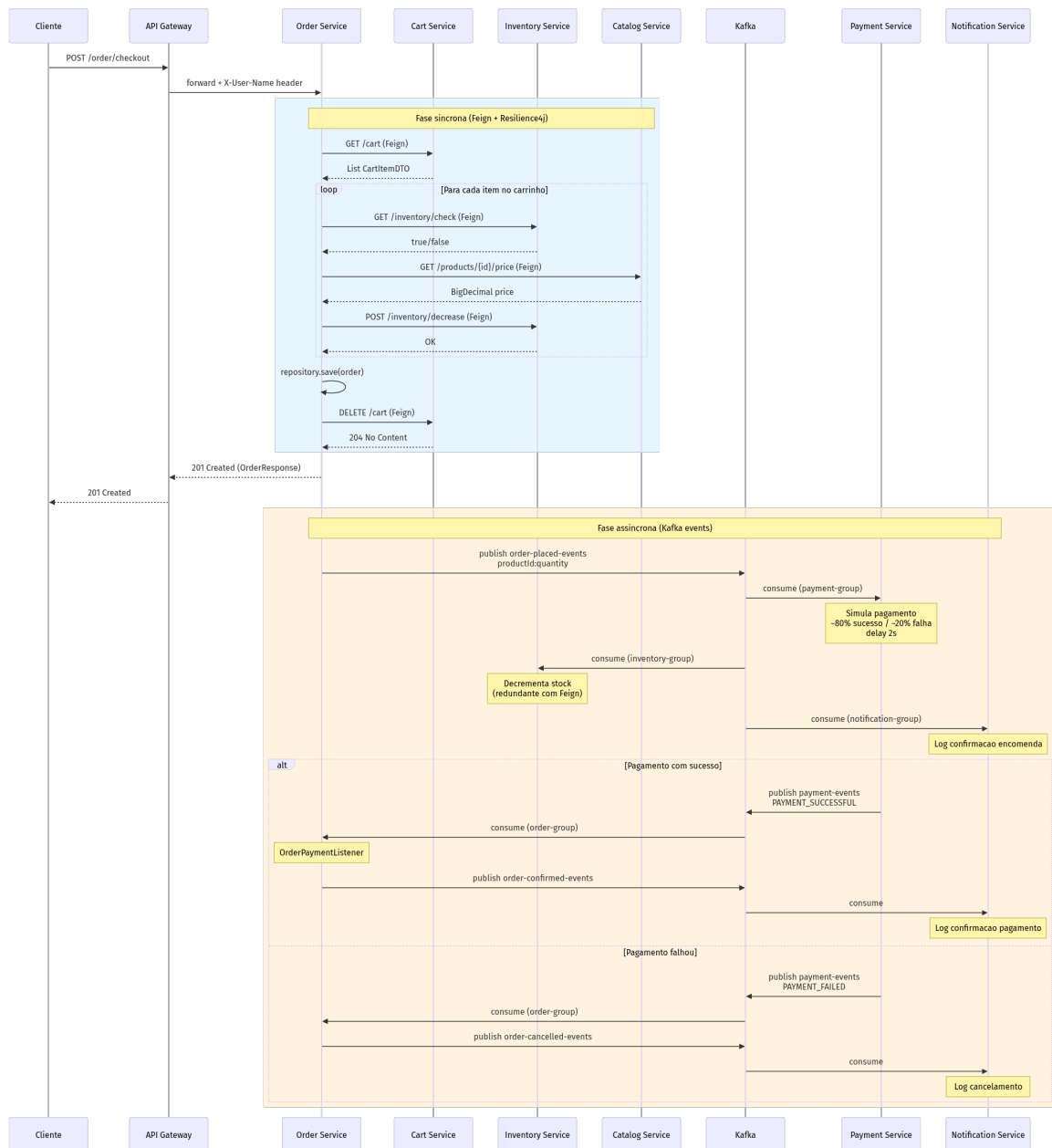
Query monólito	JOIN implícito	Solução microserviços
SELECT o FROM Order o WHERE o.user.id = ?1	Order → User (FK user_id)	Order Service filtra por username (String). Sem JOIN, sem FK.

Query monólito	JOIN implícito	Solução microserviços
OrderLine → Product (@ManyToOne)	OrderLine → Product (FK product_id)	OrderItem guarda productId, quantity e price localmente. Sem FK para o Catalog.
Order → OrderLine (@OneToMany LAZY)	Order → OrderLine (mesma BD)	Order → OrderItem (mesma BD xcommerce_orders). Este JOIN mantém-se local.
ShoppingCart continha Hashtable<Product, Integer>	Product completo serializado no Redis	CartItem guarda apenas productId + quantity. Sem referência ao Product.

Operação	Serviço chamador	Serviço chamado	Dados lidos	Mecanismo
Checkout — ler carrinho	Order Service	Cart Service	List<CartItem DTO> (productId, quantity)	Feign GET /cart
Checkout — validar stock	Order Service	Inventory Service	boolean (há stock?)	Feign GET /inventory/check
Checkout — obter preço	Order Service	Catalog Service	BigDecimal (preço atual)	Feign GET /products/{id}/price
Checkout — decrementar stock	Order Service	Inventory Service	— (escrita)	Feign POST /inventory/decrease
Checkout — limpar carrinho	Order Service	Cart Service	— (escrita)	Feign DELETE /cart
Criar produto — sync stock	Catalog Service	Inventory Service	— (escrita)	REST POST /inventory/sync

Operação	Serviço chamador	Serviço chamado	Dados lidos	Mecanismo
Registo — sync perfil	Auth Service	User Service	— (escrita)	REST POST /users/internal/sync
Criar user — sync credenciais	User Service	Auth Service	— (escrita)	REST POST /auth/internal/sync

Risco de Inconsistência



Cenário 1 — Stock (risco alto)

No monólito: O campo `Product.quantity` existia (inicializado a 5 no `data.sql`) mas nunca era decrementado no checkout. Não havia validação de stock. Inconsistência por omissão — o stock era sempre estático.

Nos microserviços: O Order Service decrementa stock de duas formas redundantes:

1. Chamada síncrona: `InventoryClient.decreaseStock(productId, quantity)` — durante o checkout

2. Evento assíncrono: topic `order-placed-events` consumido pelo `InventoryConsumer.handleOrderPlaced()` — após o checkout

Risco: Se o Order Service executa `decreaseStock()` via Feign com sucesso mas falha antes de persistir a Order (ex: timeout na BD), o stock fica decrementado sem encomenda correspondente. Não há mecanismo de compensação (rollback distribuído).

Cenário 2 — Encomendas (risco médio)

O Order Service persiste a encomenda localmente e depois publica eventos Kafka (`order-placed` + `order-placed-events`). Se o Kafka estiver indisponível no momento da publicação:

- A encomenda é criada com status `HANDLING`
- O Payment Service nunca recebe o evento → nunca processa pagamento
- A encomenda fica indefinidamente em `HANDLING`

Não existe dead letter queue nem mecanismo de retry para republicação.

Cenário 3 — Carrinho (risco baixo)

No checkout, o Order Service:

1. Lê o carrinho (`getCartItems`)
2. Processa e persiste a encomenda
3. Limpa o carrinho (`clearCart`)

Se falhar entre os passos 2 e 3, a encomenda é criada mas o carrinho mantém os items. O utilizador poderia fazer checkout novamente, criando encomenda duplicada. O fallback do `clearCart` é apenas um log warning — o checkout continua mesmo que a limpeza falhe.

Cenário 4 — Dados de utilizador duplicados (risco médio)

O User existe em duas BDs:

BD	Tabela	Campos
xcommerce_auth	users	username, email, password, role, active
xcommerce_users	users	username, email, password, first_name, last_name, address, phone, role, active

A sincronização via REST (`/internal/sync`) usa Resilience4j com retry e circuit breaker. Se o serviço destino estiver down por mais tempo que os retries, o sync falha silenciosamente (log warning). Os dados divergem até uma próxima operação de sync.

Cenário 5 — Pagamento assíncrono (risco controlado)

O Payment Service simula ~20% de falhas (`new Random().nextDouble() > 0.2`).

Se o `OrderPaymentListener` falhar ao consumir `payment-events`, a encomenda fica em estado incoerente — o pagamento foi processado mas o status da Order não é atualizado.

Resumo de Riscos

Cenário	Risco	Causa raiz	Mecanismo de mitigação existente	Lacuna
Stock inconsistente	Alto	Falha entre Feign e persist	Resilience4j (Retry + CB)	Sem compensação/roll back distribuído
Encomenda em HANDLING permanente	Médio	Kafka indisponível	Nenhum	Sem dead letter queue nem retry de publicação
Carrinho não limpo	Baixo	Falha no clearCart	Fallback: log warning,	Possível duplicação de encomenda

Cenário	Risco	Causa raiz	Mecanismo de mitigação existente	Lacuna
			checkout continua	
User dessincronizado	Médio	Serviço destino down	Resilience4j (Retry + CB)	Sem reconciliação periódica
Payment não refletido	Controlado	Consumer falha	Kafka consumer group (rebalance)	Sem idempotência explícita

Plano de Migração Incremental

A decomposição seguiu uma estratégia de **Strangler Fig** em 6 fases, cada uma extraíndo funcionalidade do monólito para um serviço independente:

Fase 1 — Auth Service

Aspeto	Detalhe
Extraído de	<code>SecurityConfigurer</code> , <code>JwtHelper</code> , <code>AuthProvider</code> , <code>UserDetailsService</code>
BD criada	<code>xcommerce_auth</code>
Mudança-chave	JWT deixa de ser validado no monólito; o API Gateway (AuthenticationFilter) assume a validação como GlobalFilter
Dependência introduzida	Auth → User Service (sync de perfil via REST)

Fase 2 — Catalog Service

Aspeto	Detalhe
Extraído de	<code>Product</code> , <code>Brand</code> , <code>Category</code> , <code>ProductRestRepository</code> , <code>BrandRestRepository</code> , <code>CategoryRestRepository</code>
BD criada	<code>xcommerce_catalog</code>
Mudança-chave	<code>@RepositoryRestResource</code> (Spring Data REST, endpoints auto-gerados) substituídos por controllers explícitos (<code>ProductController</code> , <code>BrandController</code> , <code>CategoryController</code>) com DTOs
Schema	Flyway migrations versionadas

Fase 3 — Cart Service

Aspeto	Detalhe
Extraído de	ShoppingCartRestController, ShoppingCartService, ShoppingCart (DTO)
BD criada	xcommerce_cart
Mudança-chave	Carrinho deixa de ser um DTO Redis (Hashtable<Product, Integer> com @Cacheable/@CachePut) e passa a ser entidade relacional CartItem(username, productId, quantity). Elimina dependência do Redis e do schema Product.
Identificação	userId numérico substituído por username (String, via header X-User-Name)

Fase 4 — Order Service + Inventory Service

Aspeto	Detalhe
Extraído de	OrderRestController, OrderService, Order, OrderLine
BD criadas	xcommerce_orders, xcommerce_inventory
Mudança-chave	Checkout refatorizado: transação local → orquestração via Feign (Cart → Inventory → Catalog → Inventory → persist → Cart clear). Inventory Service é completamente novo — no monólito, Product.quantity existia mas nunca era usado.
Comunicação	OpenFeign + Resilience4j (Circuit Breaker + Retry + Bulkhead)
Eventos	Kafka topics: order-placed-events, order-placed

Fase 5 — Payment, Notification e User Service

Aspeto	Detalhe
Novos serviços	Payment Service, Notification Service, User Service
BDs criadas	<code>xcommerce_payment</code> , <code>xcommerce_notification</code> , <code>xcommerce_users</code>
Payment	Processamento assíncrono via Kafka (80% sucesso simulado). Não existia no monólito — checkout criava a encomenda diretamente com status HANDLING.
Notification	Event sink que consome 6 topics Kafka. No monólito havia apenas um comentário <code>// use JavaMail API to send a notification</code> .
User	Split da tabela <code>"user"</code> original: credenciais ficam no Auth, perfil no User Service. Sync bidirecional via REST <code>/internal/sync</code> .

Fase 6 — API Gateway + Observabilidade

Aspeto	Detalhe
Gateway	Spring Cloud Gateway (porta 9000). 10 rotas de negócio + 8 de docs. AuthenticationFilter como GlobalFilter. Circuit breaker por rota com fallback 503.
Resiliência	Resilience4j em Order Service (<code>OrderRemoteService</code>), Auth Service (<code>UserSyncService</code>), Catalog Service (<code>InventorySyncService</code>)
Observabilidade	Prometheus (métricas), Grafana (dashboards), Jaeger (tracing distribuído), OpenTelemetry (instrumentação), Kafdrop (UI Kafka)

Aspeto	Detalhe
Infraestrutura	Docker Compose: 16 containers numa rede <code>xcommerce-network</code>

Metodologia Experimental e Testes Iniciais

Ambiente Experimental

Aspeto	Monólito	Microserviços
Máquina de desenvolvimento	macOS	macOS
Java	8	21
Build tool	Gradle	Maven
Base de dados	PostgreSQL local (1 BD)	PostgreSQL 15 Alpine via Docker (9 BDs lógicas)
Cache	Redis local	— (cart em BD relacional)
Message broker	—	Kafka 7.5.0 + Zookeeper (Docker)
Containers	—	Docker Compose, 16 containers
Rede	localhost	Docker bridge network <code>xcommerce-network</code> , DNS interno
Build dos serviços	<code>./gradlew bootRun</code>	Multi-stage Dockerfile (Maven build → JRE 21 Alpine)
Health checks	—	Spring Boot Actuator (<code>/actuator/health</code>) por serviço

Todos os containers correm na mesma máquina (Docker Desktop), com CPU e memória nos valores default do Docker. Não foi aplicada configuração de limites de recursos.

Ferramentas

Ferramenta	Utilização
Docker Compose	Orquestração dos 16 containers
Postman / cURL	Testes manuais de endpoints via Gateway (porta 9000)
Kafdrop (porta 9001)	Visualização de topics e mensagens Kafka
Jaeger (porta 16686)	Tracing distribuído (OpenTelemetry)
Prometheus (porta 9090)	Recolha de métricas (Actuator + Micrometer)
Grafana (porta 3000)	Dashboards de métricas
Logs Docker	<code>docker compose logs -f <serviço></code> para observar fluxos Kafka
Dashboard Node.js (porta 3001)	Frontend admin com dados mock

Cenários de Teste

ID	Cenário	Objetivo	Hipótese testada
CT1	Registo + Login + JWT	Verificar fluxo Auth completo com sync User Service	H1
CT2	Adicionar produto ao carrinho	Verificar Cart Service autónomo (sem dependência do Catalog)	H1, H2
CT3	Checkout end-to-end	Verificar orquestração Order → Cart → Inventory → Catalog + Kafka	H2, H3, H5
CT4	Consultar encomendas	Verificar leitura com dados desnormalizados (sem composição)	H2

ID	Cenário	Objetivo	Hipótese testada
CT5	Derrubar um serviço (ex: Payment)	Verificar isolamento de falhas e fallback 503 no Gateway	H1, H4
CT6	Derrubar BD do monólito	Verificar SPOF — todas as funcionalidades devem falhar	H1
CT7	Criar produto com sync Inventory	Verificar evento Kafka <code>product-events</code> + sync REST	H3
CT8	Checkout com Inventory indisponível	Verificar Circuit Breaker e fallback no Order Service	H5

Variáveis Controladas

- Número de produtos no catálogo: 4 (seed do monólito) / configurável via POST nos microserviços
- Estado inicial da BD: `data.sql` (monólito) / `Flyway V1__init` (microserviços)
- Configuração de containers: CPU e memória nos defaults do Docker Desktop
- Rede: todos os containers na mesma bridge network (`xcommerce-network`)
- Versões de dependências fixadas nos `pom.xml` (Spring Boot 3.4.0, Spring Cloud 2024.0.0, Kafka 7.5.0)
- Secret JWT: `"my-secret-key"` (hardcoded, igual no Auth Service e Gateway)

Métricas

Métrica	Como medir	Relevância
Tempo de resposta (latência)	Timestamp Postman / cURL	QI5 — overhead inter-serviço
Disponibilidade parcial	Derrubar container + testar outros serviços	QI1, QI4 — isolamento de falhas

Métrica	Como medir	Relevância
Número de chamadas de rede	Contagem no código (Feign calls) + Jaeger traces	QI2, QI5 — complexidade do checkout
Consistência de stock	Comparar Inventory após checkout vs Order criada	QI3 — riscos de inconsistência
Taxa de sucesso do checkout	Repetir checkout N vezes, contar sucesso/falha	H5 — resiliência
Circuit breaker activations	Prometheus metrics (resilience4j_circuitbreaker_*)	H4, H5
Containers independentes deployáveis	Contagem: 9 serviços × build/deploy separado	QI1 — deployability

Testes Operacionais Já Realizados

Os seguintes testes foram executados com sucesso:

- 1. Levantamento do sistema completo** — `docker compose up` levanta os 16 containers. Todos os health checks passam (`/actuator/health` retorna UP).
- 2. Registo e login** — `POST /auth/register` cria utilizador no Auth Service + sync para User Service. `POST /auth/login` retorna JWT válido. Token verificável no Gateway.
- 3. CRUD de catálogo** — Criação de produtos via `POST /products` com sync automático para Inventory (REST + Kafka `product-events`). Listagem via `GET /products` funcional.
- 4. Carrinho** — Adicionar items via `PATCH /cart/addProduct`, listar via `GET /cart`, remover via `PATCH /cart/removeProduct`. Tudo via Gateway com header `Authorization: Bearer <JWT>`.
- 5. Checkout end-to-end** — `POST /order/checkout` executa: ler carrinho → validar stock → obter preço → decrementar stock → persistir order → publicar Kafka → limpar carrinho. Logs confirmam fluxo completo.

6. **Kafka funcional** — Observados via Kafdrop e logs:

- `product-events`: Catalog → Inventory (sync stock)
- `order-placed-events`: Order → Payment + Inventory + Notification
- `payment-events`: Payment → Order + Notification
- `user-events`: Auth → Cart + Notification

7. **Dashboard admin** — Aplicação Node.js funcional como frontend de visualização.

Resultados Preliminares

Evidências Recolhidas

Evidência	Detalhe
Decomposição concluída	1 artefacto monolítico (33 classes Java, 1 BD, 1 JAR) → 9 serviços (143 classes Java, 9 BDs, 9 JARs)
Serviços independentes	Cada serviço arranca isoladamente e é testável sem os restantes (exceto dependências explícitas)
Database per Service	9 databases com schemas independentes, geridos por Flyway
Comunicação funcional	REST síncrono (Feign) e Kafka assíncrono ambos operacionais
Circuit breakers ativos	Gateway retorna fallback 503 quando serviço downstream está indisponível
Sync Auth ↔ User	Endpoints <code>/internal/sync</code> bidirecionais funcionam com Resilience4j
Pagamento simulado	Payment Service processa ~80% com sucesso, publica resultado via Kafka

Comportamento do Monólito Perante Falha da Base de Dados

Cenário: Parar o PostgreSQL enquanto o monólito está em execução.

Funcionalidade	Resultado	Razão
Login / Autenticação	FALHA	<code>SecurityConfigurer</code> faz query JDBC direta (<code>SELECT * FROM "user" WHERE username = ?</code>)

Funcionalidade	Resultado	Razão
Catálogo (GET /products)	FALHA	Spring Data REST depende de JPA → sem BD, sem resposta
Carrinho — leitura	PARCIAL	<code>@Cacheable("ShoppingCart")</code> pode retornar dados do Redis se já estiverem em cache
Carrinho — adicionar produto	FALHA	<code>ShoppingCartService.addProduct()</code> chama <code>productRepository.findById()</code> → precisa da BD
Checkout	FALHA	<code>OrderService.checkout()</code> precisa de <code>userRepository.findById()</code> e <code>orderRepository.save()</code>
Encomendas — listagem	FALHA	<code>orderRepository.findByCustomer()</code> precisa da BD

Conclusão: O monólito tem um **single point of failure completo** na BD PostgreSQL. Apenas dados previamente cacheados em Redis (carrinhos já lidos) permanecem acessíveis, e mesmo esses só suportam leitura.

Contraste com microserviços: Se a BD do Inventory Service falhar:

- Auth Service → continua funcional
- Catalog Service → continua funcional (GET /products responde)
- Cart Service → continua funcional
- Order Service → checkout falha com circuit breaker (fallback), mas listagem de encomendas existentes funciona
- Todos os outros serviços → não afetados

Testes por Workflow

WF1 — Adicionar Produto ao Carrinho

Aspeto	Monólito	Microserviços
Endpoint	PATCH /rest/shoppingCart/addProduct	PATCH /cart/addProduct (via Gateway :9000)
Chamadas de rede	0 (in-process)	0 (Cart Service opera localmente)
Acesso a BD	1 SELECT (ProductRepository)	1 SELECT/INSERT (CartRepository)
Acesso a cache	1 GET + 1 PUT (Redis)	—
Dependência de outro módulo	Sim (Product entity do Catalog)	Não (guarda apenas productId)
Resultado	Carrinho atualizado com Product completo	Carrinho atualizado com productId + quantity

WF2 — Consultar Encomendas

Aspeto	Monólito	Microserviços
Endpoint	GET /rest/order/list	GET /order/list (via Gateway :9000)
Chamadas de rede	0	0 (dados desnormalizados locais)
Queries	1 + N + M (N+1 problem com LAZY loading)	1 SELECT orders + 1 SELECT order_items (mesmo schema)
Dados retornados	Order + OrderLine + Product completo (nome, marca, categoria)	Order + OrderItem (productId, price, quantity — sem nome/marca)

Aspeto	Monólito	Microserviços
Composição	Automática via JPA/Hibernate	Não necessária (dados locais, embora incompletos)

WF3 — Checkout

Aspeto	Monólito	Microserviços
Endpoint	GET /rest/order/checkout	POST /order/checkout (via Gateway :9000)
Chamadas de rede	0	2 + 3N (para N produtos no carrinho)
Validação de stock	Não	Sim (Feign → Inventory)
Decremento de stock	Não	Sim (Feign + Kafka)
Pagamento	Não	Sim (Kafka → Payment Service, ~80% sucesso)
Notificação	Não	Sim (Kafka → Notification Service)
Transação	Atômica local (JPA)	Distribuída (sem atomicidade global)
Resiliência	Nenhuma	Resilience4j (CB + Retry + Bulkhead)

Discussão Inicial

Onde a Decomposição É Mais Sólida

Auth Service — Separação limpa. Credenciais e JWT isolados. Não depende do catálogo nem das encomendas para autenticar. Pode escalar independentemente.

Catalog Service — CRUD auto-contido de produtos, marcas e categorias. A publicação de eventos Kafka para o Inventory é fire-and-forget (não bloqueia o fluxo principal de criação de produto).

Cart Service — A migração de Redis (`Hashtable<Product, Integer>`) para BD relacional (`CartItem(username, productId, quantity)`) eliminou o acoplamento estrutural com o schema Product. É o serviço mais autônomo — não chama nenhum outro via REST.

Notification Service — Puramente event-driven. Consome 6 topics Kafka, não é chamado por ninguém via REST. Pode falhar sem impacto funcional em qualquer operação de negócio.

Onde Surgem Dificuldades

Checkout (Order Service) — Orquestra 3+ chamadas Feign síncronas + publicação Kafka. A falha de qualquer dependência bloqueia o checkout. O Resilience4j mitiga falhas transitórias (retries, circuit breaker), mas não resolve inconsistências de dados quando uma chamada remota sucede e outra falha.

Consistência de stock — O Order Service decrementa stock por duas vias: REST síncrono (`InventoryClient.decreaseStock()`) e evento Kafka (`order-placed-events`). Esta duplicação introduz risco: se uma via falha e a outra não, o stock pode ser decrementado duas vezes ou zero vezes. Uma abordagem mais robusta usaria uma única via (ex: apenas Kafka com confirmação).

Sync Auth ↔ User — Sincronização bidirecional via REST `/internal/sync`. Se a chamada falhar (serviço destino down), os dados divergem silenciosamente (fallback = log warning). Não há reconciliação periódica.

Leituras compostas — O Cart Service guarda apenas `productId`. Para mostrar nome e preço do produto no carrinho, o frontend precisaria de chamar também o Catalog Service. Este cenário não está resolvido no backend.

Acoplamento Residual

Apesar da separação em 8 serviços, existem dependências práticas:

Dependência	Tipo	Impacto
Order → Cart + Inventory + Catalog	Feign síncrono (checkout)	Order Service é um orquestrador; falha se qualquer um falhar
Auth ↔ User	REST bidirecional (sync)	Dependência circular; divergência silenciosa em caso de falha
Catalog → Inventory	REST + Kafka (sync stock)	Criação de produto depende do Inventory para sync
<code>productId</code> como chave virtual	Referência em Cart, Order, Inventory	Sem integridade referencial formal entre serviços

Este acoplamento é inerente ao domínio — não é possível eliminar a dependência entre encomendas e stock. O que mudou é que o acoplamento agora é **explícito** (chamadas Feign com circuit breaker e fallback definido) em vez de **implícito** (FK na mesma BD com falha silenciosa).

Limitações do Estudo

- Não foram realizados load tests comparativos (Locust, k6 ou JMeter) — as observações de latência são manuais
- O sistema corre em Docker Compose local, não em ambiente cloud — os resultados não refletem condições de produção
- O Payment Service é simulação (random 80/20), não testa processamento real
- O Keycloak está provisionado no Docker Compose mas não integrado — os serviços usam JWT custom do Auth Service
- Sem auto-scaling nem limites de recursos configurados nos containers
- Algumas migrações Flyway definem tabelas além do que o código utiliza (ex: `order_saga_state`, `stock_reservations`)
- Não foram estimados custos operacionais (custo por serviço em cloud)

Referências

- IRAQI, O. *XCommerce Monolithic (Spring Boot)*. GitHub. <https://github.com/oiraqi/xcommerce-monolithic>
- Newman, S. (2021). *Building Microservices: Designing Fine-Grained Systems* (2nd ed.). O'Reilly Media.
- Richardson, C. (2018). *Microservices Patterns*. Manning Publications.
- Fowler, M. (2014). "Microservices." <https://martinfowler.com/articles/microservices.html>
- Spring Cloud Gateway Documentation. <https://docs.spring.io/spring-cloud-gateway/>
- Resilience4j Documentation. <https://resilience4j.readme.io/docs>
- Apache Kafka Documentation. <https://kafka.apache.org/documentation/>
- Docker Documentation. <https://docs.docker.com/compose/>
- Spring Boot 3.4 Reference. <https://docs.spring.io/spring-boot/docs/3.4.0/reference/htmlsingle/>
- OpenTelemetry Documentation. <https://opentelemetry.io/docs/>